FitStyle AI

AI-Powered Fashion Fitting Platform with Cloud Computing

	Student Name	ID
1	Afnan Mohamed Ali Dior	20596319
2	Alaa Ehab Mohamed El Shenawy	20596313
3	Hind Alaa Fathy Abdelhalim Ahmed	20596337
4	Manar Samir Mohamed Elabsi	20596305

Program:

M.S. in Computer Science

Supervisor/Advisor:

Dr. Anwar Hossain

Date Submitted:

20/10/2026

Abstract

What if you could try on an outfit without leaving your screen? Online shoppers still guess their size, guess which pieces go together and guess how a garment will look on them, and wrong guesses end in returns and lost confidence. FitStyle AI answers these guesses in one shopping workflow. From a single full-body photograph and her height, a body-measurement model estimates shoulder, waist and hip widths, a body shape and a clothing size. An outfit-recommendation engine then proposes five complete outfits for her size, body shape, occasion and season, offers only items that are in stock in her size, and explains each choice in plain language. The shopper can swap any piece, preview the look on her own photograph with the LaDI-VTON virtual try-on model, build a cart, and reserve or pay for it. The web application runs locally, its database and authentication use cloud services within free-tier limits, and the try-on model runs as a service on a free GPU notebook runtime. The recommendation engine, the most data-driven component, was evaluated against catalogue-derived labels: a rebalanced compatibility model reached 0.789 held-out accuracy, up from 0.646, and a per-item reuse cap raised the number of distinct items in the top five outfits from 8 to 14. For body measurement, replacing landmark distances with silhouette widths corrected an implausible hip estimate on a test photograph. The try-on model was assessed only qualitatively, and generating an outfit takes minutes on a free GPU. No user study was conducted, so claims about shopper preference remain open. The project shows that measurement, recommendation and preview can be combined in one low-cost application.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Anwar Hossain, for his guidance, constructive feedback and encouragement throughout this project. His comments shaped both the technical direction and the way in which the work is reported.

We thank Queen's University and the Digilians programme for the teaching, the resources and the learning environment that made this work possible, and our instructors and colleagues for the discussions that helped us refine our ideas.

We also thank the Ministry of Communications and Information Technology, the Military Academy and Capgemini for the training programme and for the opportunity to carry out this project, and Eng. Farida Shaheen from Capgemini, who accompanied us throughout the project, for her guidance and support.

We are grateful to the open-source community and to the providers of the free services on which the platform is built, including Amazon Web Services, Hugging Face, Supabase and Groq, as well as to the authors of the public research and datasets cited in this report. Their openness made it possible to build a complete system at almost no cost.

Table of contents

1. Introduction 8
1.1 Background 8
1.2 Problem Statement 8
1.3 Motivation 8
1.4 Project Overview 8
1.5 Objectives 9
1.6 Research Questions 9
1.7 Scope and Limitations 10
1.8 Contributions 10
1.9 Report Organisation 10
2. Background and Literature Review 11
2.1 Size and Fit in Online Fashion 11
2.2 Vision–Language Representations for Fashion 12
2.3 Outfit Compatibility Learning 12
2.4 Outfit Recommendation Systems in Practice 12
2.5 Model Combination, Branch Balance and Diversity 12
2.6 Sensitivity of Weighted-Sum Rankings 13
2.7 Virtual Try-On 13
2.8 Summary of Related Work and Research Gap 14
3. Methodology 17
3.1 Overall Pipeline 17
3.2 Model 1: Body Measurement and Size Estimation 17
3.3 Model 2: Outfit Recommendation Engine 20
3.4 Model 3: Virtual Try-On 23
3.5 Evaluation Methodology 24
3.6 Development Approach, Tools and Technology Choices 25
4. Design and Implementation 26
4.1 Architecture Overview 26
4.2 Data Design 27
4.3 Recommendation Service 27
4.4 User Interface 28
4.5 Integration of the AI Models 30

4.6 Security and Cost Considerations ... 32
4.7 Implementation Challenges ... 32
4.8 Platform Features ... 33
5. Results and Evaluation ... 35
5.1 Evaluation Protocol and Data ... 35
5.2 Body-Measurement and Size Model (Model 1) ... 35
5.3 Outfit Recommendation Engine (Model 2) ... 36
5.4 Virtual Try-On Model (Model 3) ... 38
5.5 Summary and Threats to Validity ... 41
6. Discussion ... 43
6.1 Interpretation of the Compatibility Results ... 43
6.2 Stability and Design Choices ... 43
6.3 Trade-offs of Pre-Computation ... 43
6.4 Relation to Existing Work ... 43
6.5 Fairness, Inclusivity and Ethical Considerations ... 44
6.6 Were the Objectives Met? ... 44
6.7 Discussion of Models 1 and 3 and of the Integrated System ... 44
7. Conclusion and Future Work ... 46
7.1 Conclusion ... 46
7.2 Future Work ... 46
References ... 48
Appendices ... 50
Appendix A. System Architecture and Recommendation Interfaces ... 50
Appendix B. Recommendation Pipeline Configuration ... 50
Appendix C. Pipeline Summary ... 51
Appendix D. Catalogue Description ... 51
Appendix E. Body-Shape and Occasion Prompts ... 52
Appendix F. Reproducibility ... 52
Appendix G. Appendices for Models 1 and 3 ... 53
Appendix H. Application Screens ... 54
Appendix I. MediaPipe Pose Landmarks ... 61
Appendix J. Body-Measurement Service Interface ... 62
Appendix K. Virtual Try-On Service Interface ... 62

List of Tables and Figures

Table 1.1. Main components of FitStyle AI. 9

Table 2.1. Summary of the outfit-recommendation literature cited in this report. 14

Table 2.2. Distinguishing properties of the outfit-recommendation approaches. 14

Table 2.3. Approaches related to the body-measurement and virtual try-on components. 15

Figure 3.1. End-to-end flow of a shopper through FitStyle AI. 17

Figure 3.2. Pipeline of the body-measurement model. 18

Figure 3.3. Pipeline of the outfit recommendation engine (A: offline preparation, B: online use). 20

Figure 3.4. Pipeline of the virtual try-on model (Model 3) and the stages added around LaDI-VTON. 23

Table 3.1. Tools and technology choices. 25

Figure 4.1. System architecture of FitStyle AI. 26

Table 4.1. Layers of the FitStyle AI implementation. 26

Figure 4.2. Try-on step: the recommended outfit (left), the preview of the shopper (centre) and the shopper's selection (right). 28

Figure 4.3. Panels of the try-on step: (a) recommended outfit, (b) all five outfits and (c) shopper's selection. 29

Figure 4.4. Dialog listing the tops that suit the shopper's profile; choosing one replaces the current top. 30

Figure 4.5. Explanation card ("Why this outfit") shown below the preview. 30

Figure 4.6. Body-measurement step: extracted metrics with an editable size selector (left) and the annotated photograph (right). 31

Table 4.2. Integration challenges and the measures taken. 32

Table 4.3. Features of the application. 33

Table 5.1. Hip-width estimate for one test photograph: landmark baseline and segmentation-based method. 35

Table 5.2. Behaviour of the hybrid waist detection across garment types. 35

Table 5.3. Held-out accuracy of the compatibility models. 36

Figure 5.1. Feature ablation: test accuracy with visual features only, metadata only, both (linear model) and with the CompatibilityNet v2. 36

Figure 5.2. Validation accuracy for different weights on the CompatibilityNet (w = 0 is the linear model alone). 37

Figure 5.3. Effect of the per-item reuse cap on the number of distinct items (a) and on set-level scores (b). 37

Figure 5.4. Top five outfits returned by the notebook for a sample profile. 38

Table 5.4. Try-on runs recorded in the notebook and in the website. 39

Figure 5.5. Try-on from the website: (a) the shopper's photograph while the try-on is being generated and (b) the result, in which the top of the selected outfit is rendered on the shopper. 41

Figure 5.6. Try-on runs in the notebook: (a) a dress; (b) trousers and a T-shirt from 60 × 80 pixel catalogue images. Each row shows the shopper's photograph, the garment image and the result; in (b) the trousers are applied first and the T-shirt second. 40

Table 6.1. Status of the project objectives. 44

Table A.1. Recommendation interfaces. 50

Table B.1. Parameters of the recommendation pipeline. 50

Table D.1. Catalogue composition by garment category. 51

Table E.1. Silhouette preferences used for the body-shape score. 52

Table G.1. Parameters of the body-measurement model (from the notebook and the service file). 53

Table G.2. Parameters of the virtual try-on model and service (from the notebook). 53

Figure H.1. Landing page. 54

Figure H.2. Authentication screens: (a) shopper sign-in, (b) store-owner sign-in, (c) shopper sign-up and (d) store-owner sign-up, which is by invitation only. 55

Figure H.3. Step 1: upload of the photograph and choice of the occasion. 55

Figure H.4. Step 4: creation of the order, listing the chosen pieces by category slot, with export of a PDF summary. 56

Figure H.5. Shopping cart with the selected pieces, the order summary, a promo-code field and the choice between buying online and paying offline. 57

Figure H.6. Step 5: payment in the sandbox gateway with a test card. 57

Figure H.7. Step 6: order confirmation with delivery tracking and the download of the PDF summary. 58

Figure H.8. PDF summary of an order paid offline, with the QR code and the shop location. 59

Figure H.9. Store owner's inventory dashboard. 59

Figure H.10. Store owner's sales and analytics dashboard. 60

Figure H.11. Try-on step after generation: the preview shows the shopper wearing the white top of the selected outfit, and the status label reads "Virtual Fitting Active". 61

Table I.1. The 33 pose landmarks and their use in the body-measurement model. 61

Table J.1. Request and response of the body-measurement service (from the service file). 62

Table K.1. Interface of the virtual try-on service (from the notebook). 62

1. Introduction

1.1 Background

Online fashion retail lets shoppers browse thousands of products from home, but it lacks two things that a physical fitting room provides: confirmation that a garment will fit the wearer, and the chance to see how pieces look together on her. Han et al. observe that the ubiquity of online fashion shopping demands effective recommendation services for customers [4]. Fit is a commercial as well as a personal problem: Sheikh et al. note that predicting the correct fit improves customer satisfaction and reduces the cost of size-related returns [13]. Style is a second source of uncertainty: Koukopoulos et al. argue that outfit recommendations simplify decision-making, increase consumer confidence and raise conversion rates [7]. At the same time, advances in computer vision, vision–language models and managed cloud services make it feasible to offer these capabilities inside an ordinary web shop.

1.2 Problem Statement

A shopper who buys clothes online faces three connected problems. (i) **Size uncertainty:** size charts differ between brands and shoppers rarely know their own measurements, so they order several sizes or return items. (ii) **Outfit uncertainty:** shops usually present products one by one, leaving the shopper to decide which garments go together for a given occasion and body shape. (iii) **Lack of preview:** the shopper cannot see a garment on herself before buying. These problems are normally handled by separate tools, if at all, so the shopper must combine several sources of advice by herself. The problem addressed by this project is how to bring measurement, recommendation and preview together in one low-cost shopping application whose database and authentication use cloud services.

1.3 Motivation

A single workflow that starts from one photograph can remove manual steps for the shopper and give the shop fewer wrong-size purchases and a more engaging experience. The project is also motivated by a practical engineering question: whether a complete application with three AI components, user accounts, a product catalogue, a cart, checkout and administrator tools can be built by a small team using the free tiers of managed cloud services for data, authentication and storage, with the cloud resources defined as code so that they can be reproduced.

1.4 Project Overview

FitStyle AI is an AI-powered fashion fitting platform with cloud computing. The shopper creates an account, uploads a full-body photograph, reviews the body measurements and clothing size that the system infers, chooses an occasion, receives recommended outfits, previews garments on her photograph, and adds the selected items to a cart and pays. Administrators manage the product catalogue and view sales analytics. The front end is a React single-page application and the back end is a Node.js/Express server; both run locally on the team's machine and are not hosted in the cloud. The cloud is used for identity and data only: authentication uses Amazon Cognito, the catalogue, user profiles, carts, measurements and orders are stored in Amazon DynamoDB, product images are kept in Supabase Storage, and the AWS resources are defined with Terraform. Three AI components are integrated into the shopper's journey (Table 1.1 and Figure 3.1).

Table 1.1. Main components of FitStyle AI.

Component	Purpose	Documented in
Web platform: React front end, Express server, Cognito, DynamoDB, Terraform	Accounts, catalogue, cart, checkout, orders, administrator tools	Sections 3.6 and 4
Model 1: body measurement and size estimation	Infers body measurements, body shape and clothing size from a photograph	Sections 2.1, 3.2, 4.5, 5.2 and Appendix G
Model 2: outfit recommendation engine	Recommends five complete outfits for the shopper's size, body shape, occasion and season, with an explanation	Sections 3.3, 4.3, 5.3 and the Appendices
Model 3: virtual try-on	Renders the selected top, bottom or dress on the shopper's photograph with the LaDI-VTON latent-diffusion try-on model	Sections 2.7, 3.4, 4.5, 5.4 and Appendices G, H and K

The output of Model 1 is the input of Model 2: the body shape and size inferred from the photograph, together with the occasion chosen by the shopper, define the profile for which outfits are recommended. The selected outfit then supplies the garment image that Model 3 renders.

1.5 Objectives

The objectives of the project are as follows.

- **O1. Platform.** To build a secure web application, whose data and authentication are handled by cloud services, with user accounts, a product catalogue, a shopping cart, checkout and order history, and administrator tools for product management and sales analytics.
- **O2. Body measurement and size.** To estimate body measurements, body shape and clothing size from a single full-body photograph (Model 1).
- **O3. Outfit recommendation.** To recommend complete outfits that suit the shopper's size, body shape, occasion and season, using only products that are in stock in her size, and to explain each recommendation (Model 2).
- **O4. Virtual try-on.** To let the shopper preview a selected garment on her own photograph (Model 3).
- **O5. Integration.** To connect the three models and the platform into one workflow in which the output of each stage is the input of the next.
- **O6. Low cost and reproducibility.** To design the system so that the cloud services it uses (a database, authentication and image storage) stay within free-tier limits, and to define the AWS resources as code.
- **O7. Evaluation.** To evaluate each AI component and the integrated system with stated metrics, and to report limitations openly.

1.6 Research Questions

The project addresses four research questions. RQ1: how accurately can body measurements, a body shape and a clothing size be inferred from a single photograph and the shopper's stated height? RQ2: can a catalogue-grounded recommendation engine compose complete outfits that suit a shopper's size, body shape, occasion and

season, remain diverse, and use only products that are in stock in her size? RQ3: how much does a learned pairwise compatibility model add over simple baselines when its labels are derived from catalogue attributes? RQ4: can a commerce platform and three AI components be integrated using only the free tiers of cloud services for data, authentication and storage?

1.7 Scope and Limitations

The project covers the full shopper journey from photograph to payment for a single shop with a catalogue of 11,513 products drawn from a public dataset [14]. The catalogue stocks sizes S to XL only. The AI components are evaluated on the data described in Section 5; no user study has been conducted, and the compatibility labels used by the recommendation engine are derived from catalogue attributes rather than human judgements, so its accuracy figures measure agreement with that proxy and not real shopper preference (Section 5.5). Payment is simulated for the purposes of the project. The body-measurement model uses a single front-facing photograph and a stated height, and its behaviour has so far been examined on a few photographs without tape-measure ground truth (Section 5.2).

1.8 Contributions

The contributions of the project are: (i) an integrated platform that combines body measurement, outfit recommendation and virtual try-on with account management, catalogue, cart and checkout; (ii) a low-cost design in which only the database, authentication and image storage are cloud services on free tiers, defined as code, and the recommendations are pre-computed; and (iii) the three AI components themselves. For the recommendation engine, the contributions are a capacity-rebalanced compatibility network with a validation-selected ensemble, a beam-search outfit construction with a per-item reuse cap and a compatibility floor, and an end-to-end evaluation across several profiles (Section 5.3). For Model 1, the contribution is a transparent measurement pipeline in which two experimentally diagnosed weaknesses of a landmark-based baseline, the hip width and the waist under loose clothing, are corrected, and in which the reliability of every waist estimate is reported together with its value; for Model 3, the contribution is the integration of the open LaDI-VTON model into the shop workflow, through a garment-cleaning stage that makes catalogue photographs usable, a layered procedure for outfits made of a top and a bottom, the restoration of the shopper's face, hair and shoes after generation, and a queued GPU service that the website calls (Sections 3.4 and 4.5).

1.9 Report Organisation

Section 2 reviews the background and related literature. Section 3 describes the methodology, including the tools and technology choices, and Section 4 the design and implementation of the platform. Section 5 presents the results and evaluation, Section 6 the discussion, and Section 7 the conclusion and future work. References and Appendices follow.

2. Background and Literature Review

2.1 Size and Fit in Online Fashion

Fit is a central source of friction in online fashion. Sheikh et al. propose a deep-learning, content-collaborative approach to personalised size and fit recommendation and report that it improves over previously published results on two public datasets [13]. Such systems infer fit from purchase history and article data. FitStyle AI instead estimates body measurements directly from a photograph, so that a shopper with no purchase history can still receive a size.

Estimating body measurements from a single photograph combines three sub-problems: locating the body, delimiting its contour, and recovering absolute scale. For locating the body, BlazePose is a lightweight convolutional network that infers 33 body keypoints and was designed for real-time inference on mobile devices [16]. The MediaPipe Pose Landmarker used in this project follows the same 33-landmark scheme, returns landmarks in image and in world coordinates, and can optionally return a segmentation mask that gives, for each pixel, the likelihood of belonging to the detected person [22]. The distinction between the two outputs matters for clothing measurements: landmarks such as the hips mark internal joint centres, not the external contour on which garments are sized, so a distance between landmarks is not a body width. A segmentation mask, in contrast, can be read at any height and therefore gives widths at the shoulders, chest, waist, hips and thigh.

A single image has no absolute scale. Günel et al. note that recovering a person's height from one image is important for virtual garment fitting but inherently ambiguous without camera information, and report as their central finding that height can only be estimated with large uncertainty [21]. FitStyle AI therefore does not infer the shopper's height; it asks for it and uses it as the reference length.

The measurement module started from a landmark-based baseline in which each body dimension is computed as the Euclidean distance between two landmarks and converted to centimetres with a factor derived from the person's known height. Section 3.2 describes two limitations of this baseline that were found experimentally and the corrections that were made. Finally, multimodal language models offer an alternative route: Qwen2.5-VL can localise objects and interpret images [17], and the project keeps such a model as a fallback when the measurement service is unavailable. A language model provides no geometric guarantee, so its output is treated as an estimate that must be validated by rules.

Learning-based methods estimate body measurements directly. Ruiz et al. train a Body Measurement network on silhouette images of real subjects, augment its training data with an adversarial body simulator built on a parametric body model, and release BodyM, a dataset of frontal and lateral silhouettes with height, weight and measurements derived from 3D scans; their setting constrains the pose and uses height and weight as inputs [23]. This method needs training data and two views; the geometric approach of this project needs no training data and a single view, at the price of the limitations discussed in Section 6.7.

Body-shape classification is a separate problem. The Female Figure Identification Technique (FFIT) distinguishes nine shapes (hourglass, bottom hourglass, top hourglass, spoon, rectangle, diamond, oval, triangle and inverted triangle) from girth measurements, which is an advance on earlier classifications based on 2D widths [24]. Parker et al. show, however, that slight differences in the definition of the measurements can place the same body in different FFIT shapes in up to 40% of cases, and that FFIT omits the shoulder measurements that matter for

garment construction [24]. The definitions themselves are standardised in ISO 8559-1, which describes the anthropometric measurements used for clothing, for example the hip as a horizontal girth at the greatest projection of the buttocks [24], [25]. Two consequences follow for this project: a width read from a front-facing silhouette is not a standard clothing measurement, and a body-shape label obtained from it should be treated as approximate.

2.2 Vision–Language Representations for Fashion

Contrastive language–image pre-training (CLIP) learns a joint embedding space for images and text from a very large collection of image–text pairs and enables zero-shot transfer to new tasks by referring to visual concepts in natural language [1]. Chia et al. adapted this idea to retail with FashionCLIP, a CLIP-like model trained on a fashion catalogue, and released its weights [2], [15]. The present system uses Marqo-FashionCLIP, a fashion-domain model fine-tuned from a ViT-B-16 CLIP checkpoint using Generalised Contrastive Learning, which trains on categories, styles, colours and materials in addition to product descriptions [3]. Its embeddings are used here frozen, and natural-language prompts about occasion and silhouette are embedded in the same space as the product images. This is a different checkpoint from the one described by Chia et al.; the two are related by approach rather than identical.

2.3 Outfit Compatibility Learning

Han et al. formulated outfit compatibility as a sequence-modelling problem, treating an outfit as an ordered sequence of items and training a bidirectional LSTM jointly with a visual–semantic embedding on a newly collected Polyvore dataset [4]. Vasileva et al. showed that learning image embeddings that respect item type improves outfit-compatibility prediction and fill-in-the-blank tasks over the state of the art at the time [5]. More recently, Kalashi and Teimourpour propose a hybrid framework that introduces an “outfit token” to represent the global context of an outfit and addresses compatibility prediction and complementary-item retrieval together [6]. These works rely on large outfit corpora with human-curated positive outfits. FitStyle AI has no such corpus; it therefore learns a pairwise compatibility function from catalogue-derived labels and incorporates item category explicitly, in the spirit of type-aware modelling [5].

2.4 Outfit Recommendation Systems in Practice

TrendGen is an outfit recommendation and display system deployed on a major e-commerce platform; it extracts product attributes from images and titles and ranks products by aesthetic similarity as well as repetitiveness, so that generated outfits are cohesive and varied [7]. Loom combines slot-constrained nearest-neighbour retrieval over FashionCLIP embeddings with a multi-objective score covering embedding similarity, colour harmony, formality, occasion coherence, style direction and within-outfit diversity, and introduces “vibe/anti-vibe” occasion priors that embed prose descriptions of an occasion as anchor vectors in CLIP space [8]. The occasion-affinity component of the present engine follows the same general idea: an occasion is represented by a descriptive prompt and by a contrasting prompt, and products are scored by their differential similarity to the two.

2.5 Model Combination, Branch Balance and Diversity

Cheng et al. introduced Wide & Deep learning, which jointly trains a linear model and a deep network to combine the memorisation of feature interactions with the generalisation of embeddings [9]. Meng et al. study branch

imbalance in dual-branch multimodal networks and show that an imbalance between branches degrades downstream performance [10]; this motivated the rebalancing of the visual and metadata branches of the compatibility network used here (Section 5.3). For constructing sets of results, Vijayakumar et al. show that standard beam search tends to return sequences that differ only slightly, and propose Diverse Beam Search, which optimises a diversity-augmented objective [11]. The present engine addresses the analogous problem at outfit level with a per-item reuse cap.

2.6 Sensitivity of Weighted-Sum Rankings

Outfit scores in this work are weighted sums of relevance and compatibility. O'Shea et al. develop a closed-form method for computing weight stability intervals for the weighted sum model, that is, the range of a criterion weight over which the ranking of alternatives does not change [12]. Following this principle, the sensitivity of the recommended outfit set to its weights is measured and reported (Section 5.3.5) rather than assumed.

2.7 Virtual Try-On

Image-based virtual try-on synthesises an image of a person wearing a garment that is shown on its own, or on another model, in a product photograph. Early methods such as VITON predict a coarse try-on image from a person representation and the clothing item and then refine it [18]. Progress was limited by data: public datasets concentrated on upper-body garments at low resolution. Dress Code addressed this with 53,795 image pairs at 1024 × 768 pixels in three categories, upper body, lower body and dresses [26], which are the garment categories of a fashion shop.

Recent work builds on diffusion models. Latent diffusion models generate images in the compressed latent space of an autoencoder instead of pixel space [27], which makes high-resolution synthesis affordable, and their inpainting variants complete a masked region of an image under a condition. IDM-VTON adds garment-encoding modules to a diffusion model so that the details of the garment are preserved in unconstrained photographs [19]. LaDI-VTON, the model used in this work, treats try-on as latent-diffusion inpainting of the clothing region and adds two components [20]: an additional autoencoder module with learnable skip connections, which helps to preserve the characteristics of the person, and a textual-inversion component, which maps the visual features of the garment into the CLIP token-embedding space as pseudo-word embeddings that condition the generation. Its authors report that it outperforms competing methods on the Dress Code and VITON-HD datasets by a consistent margin [20].

LaDI-VTON was chosen as Model 3 for three reasons. Its code and trained weights are public [28], so it can be run without a per-call quota or the availability of an external provider. Its Dress Code weights cover tops, bottoms and dresses, the categories on which the shop's catalogue and Model 2 operate. And it needs only the garment image as a condition, so no text description or 3D model of the products is required. Its limitations are those of the approach: it needs a GPU, generating one image takes minutes rather than seconds (Section 5.4), its inputs follow the Dress Code convention (pose keypoints, body-part labels and a clean garment image) that ordinary shop photographs do not provide, and it renders clothing only, so shoes and accessories cannot be tried on. Table 2.3 places it among the related approaches.

2.8 Summary of Related Work and Research Gap

Table 2.1 summarises the works discussed above. The descriptions follow each work's published abstract; because the works address different tasks and use different datasets and metrics, the comparison is qualitative.

Table 2.1. Summary of the outfit-recommendation literature cited in this report.

Work	Task	Core technique	Relation to this work
Han et al. [4], 2017	Outfit compatibility and generation	Bi-LSTM over outfit sequence with visual–semantic embedding	Outfit-level compatibility; here pairwise scoring is used because no outfit corpus exists
Vasileva et al. [5], 2018	Type-aware compatibility	Embeddings that respect item type	Motivates category-aware compatibility features
Kalashi & Teimourpour [6], 2025	Compatibility prediction and complementary-item retrieval	"Outfit token" for global outfit context	Stronger setting with large outfit data; not reproduced here
Koukopoulos et al. [7], 2026	Deployed outfit recommendation and display	Attribute extraction; ranking by similarity and repetitiveness	Analogous to the per-item reuse cap
Berlia [8], 2026	Outfit recommendation	FashionCLIP retrieval; multi-signal score; vibe/anti-vibe priors	Similar occasion-prior idea; different search method
Cheng et al. [9], 2016	App recommendation	Jointly trained linear and deep models	Here combined post hoc with a validation-tuned weight
Meng et al. [10], 2026	Multimodal sentiment analysis	Dual-branch rebalancing	Supports the branch-rebalancing rationale (non-fashion domain)
Vijayakumar et al. [11], 2018	Sequence decoding	Diverse Beam Search	Motivates diversity beyond plain beam search
O'Shea et al. [12], 2026	Multi-criteria decision analysis	Weight stability intervals	Basis for the weight-sensitivity analysis

Source: abstracts and publication records of the cited works [4]–[13].

Table 2.2. Distinguishing properties of the outfit-recommendation approaches.

Work	Outfit-level	Occasion-aware	Body shape and size	Learned compatibility	Diversity control	Live shop
Han et al. [4]	Yes	No	No	Yes	No	No
Vasileva et al. [5]	Yes	No	No	Yes	Implicit	No
Kalashi & Teimourpour [6]	Yes	No	No	Yes	No	No
Koukopoulos et al. [7]	Yes	No	No	Yes	Yes	Yes
Berlia [8]	Yes	Yes	No	No	Yes	Yes
Sheikh et al. [13] (size and fit only)	No	No	Size only	Yes	No	No

Work	Outfit-level	Occasion-aware	Body shape and size	Learned compatibility	Diversity control	Live shop
This work	**Yes**	**Yes**	**Yes**	**Yes (proxy labels)**	**Yes**	**Yes (project scale)**

Source: compiled by the authors from the full texts of the cited works [4]–[8] and [13]. No = the property is not part of the method or evaluation described in the work; Implicit = diverse outputs are mentioned but no mechanism controls them.

In Table 2.2, each entry was assigned from the full text of the cited work. “No” means that the property is not part of the method or evaluation described there, and “Implicit” means that diverse outputs are mentioned without a mechanism that controls them. “Live shop” means that the system is described as operating inside a running online shop. A further property of this work, enforcing that every recommended piece is in stock in the shopper’s size, is not included in the comparison.

Table 2.3. Approaches related to the body-measurement and virtual try-on components.

Approach	Task	Input	Technique	Distinguishing property
BlazePose [16]	Pose estimation	Image	Lightweight network, 33 keypoints	Real-time on mobile devices; gives landmarks, not body sizes
BMnet and BodyM [23]	Body measurement	Frontal and lateral silhouettes, height, weight	Network trained with adversarial augmentation	Learned and accurate, but needs training data and two views
FFIT [24]	Body-shape classification	Girth measurements	Rule-based, nine shapes	Classification sensitive to measurement definitions
Landmark-distance baseline	Body measurement	Image and known height	Landmark distances scaled by height	Baseline of this work; joint-centre distances are not body widths
Günel et al. [21]	Height estimation	Single image	Learned from pose and face cues	Height is estimated with large uncertainty, so it is requested from the shopper here
Qwen2.5-VL [17]	Multimodal understanding	Image and text prompt	Vision–language model	General purpose; no metric guarantee; used here as a fallback
Sheikh et al. [13]	Size and fit prediction	Purchase and article data	Deep content–collaborative model	Needs purchase history; this work needs a photograph and a height
VITON [18]	Virtual try-on	Person and product image	Coarse-to-fine image generation	Early image-based method without 3D information
IDM-VTON [19]	Virtual try-on	Person and garment image, text	Diffusion with garment modules	Preserves garment detail in unconstrained images; needs a GPU

Approach	Task	Input	Technique	Distinguishing property
LaDI-VTON [20]	Virtual try-on	Person photograph, garment photograph, pose keypoints and body-part labels	Latent-diffusion inpainting with textual-inversion garment tokens and an extra autoencoder	Used by this work (Model 3); open code and weights, three garment categories; needs a GPU and minutes per image

Source: compiled by the authors from the cited works.

Three observations frame the contribution. First, the closest systems [7], [8] are evaluated on their own catalogues and metrics, so absolute figures are not comparable with those reported here. Second, no cited system combines body shape, size availability, occasion and season as explicit inputs, which is the setting of an online shop that must recommend only what the shopper can actually buy in her size. Third, the work is methodological rather than algorithmic: it applies established techniques within a diagnosis-driven process and reports negative findings openly.

3. Methodology

3.1 Overall Pipeline

A shopper's journey through FitStyle AI has four stages. (1) The shopper uploads a full-body photograph. (2) Model 1 estimates body measurements, a body shape and a clothing size. (3) The shopper selects an occasion, and Model 2 returns five complete outfits for the resulting profile (size, body shape, occasion, season). (4) The shopper selects an outfit, may replace individual garments, and previews the selected garments on her photograph with Model 3 (LaDI-VTON). The interface between the models is therefore a small, discrete profile: Model 1 supplies body shape and size, and Model 2 consumes them together with the occasion and season (Figure 3.1, in which the shaded boxes are the three AI components and the white boxes are the data passed between them).

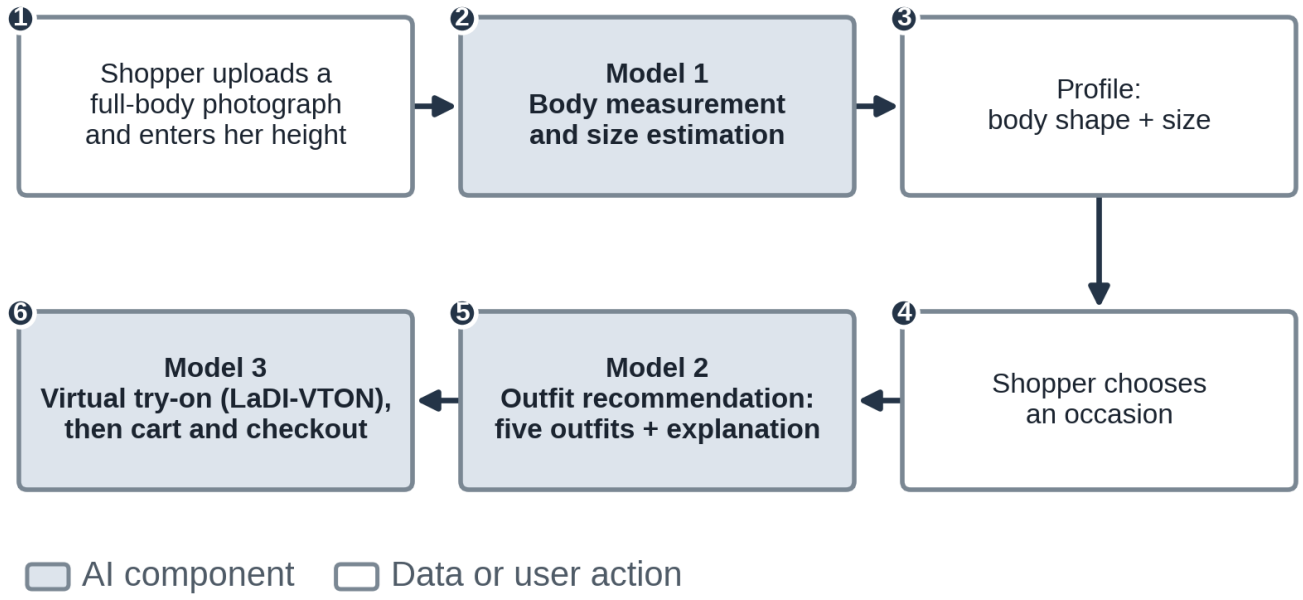


Figure 3.1. End-to-end flow of a shopper through FitStyle AI.

3.2 Model 1: Body Measurement and Size Estimation

3.2.1 Design rationale and overview

The module was developed from a landmark-based baseline and improved through two experimental diagnoses. The first concerned hip width: the distance between the two hip landmarks does not measure the width of the hips as worn, because those landmarks mark joint centres (Section 5.2). The second concerned the waist: a segmentation-based search for the narrowest point of the torso is correct for fitted clothing but fails for loose or untucked garments (Section 5.2). The final pipeline, called detect_size(), keeps the landmark distance only where the landmarks lie on the body surface (the shoulders), reads all other widths from the silhouette mask, protects the waist estimate with a plausibility check, and reports how each waist value was obtained. Figure 3.2 summarises the pipeline.

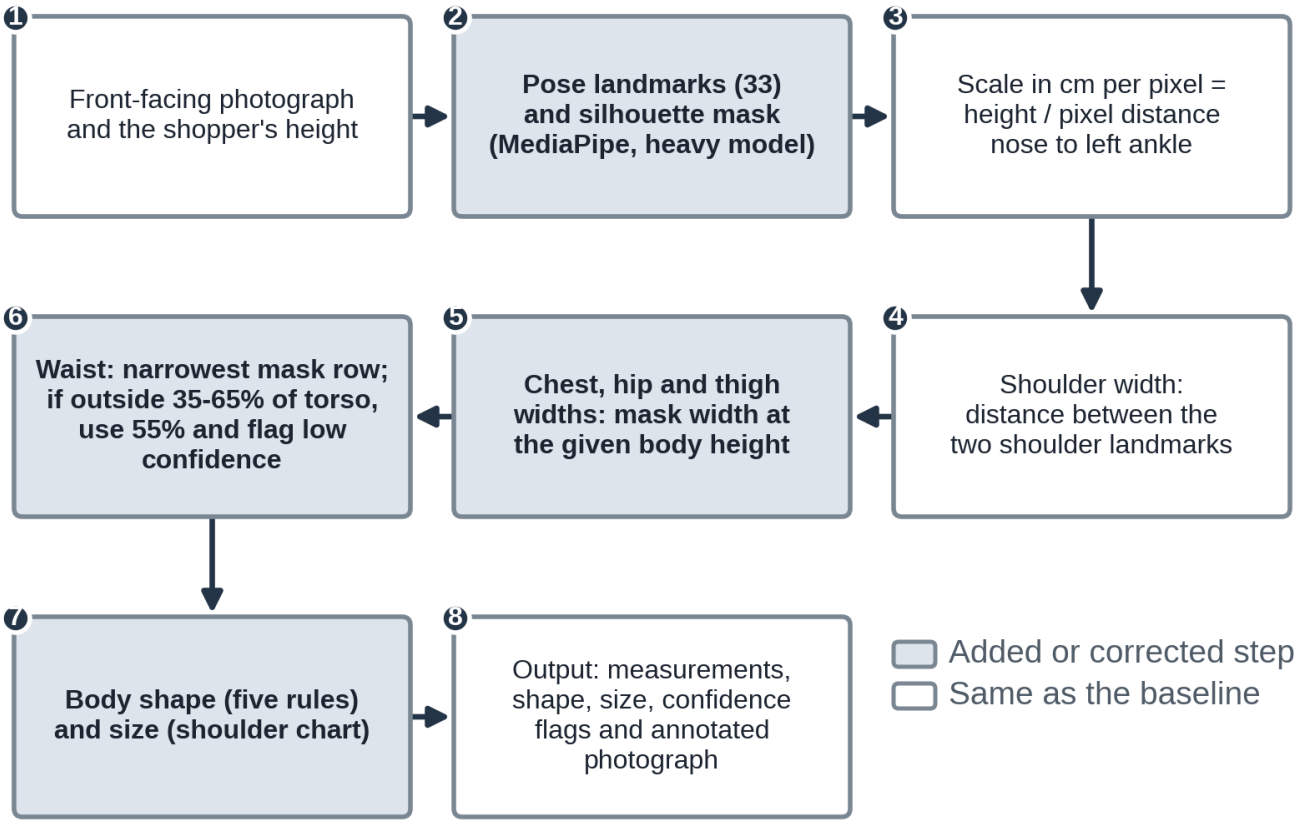


Figure 3.2. Pipeline of the body-measurement model.

3.2.2 Input and scale

The input is a front-facing, full-body photograph and the shopper's height H in centimetres (165 cm is used if none is supplied). The photograph is decoded and, because of a documented MediaPipe fault that aborts the process when the width of the segmentation mask is not a multiple of four pixels, its right edge is padded by replicating the border until the width is divisible by four; replication avoids distorting the silhouette. Pose landmarks and a segmentation mask are obtained from the MediaPipe Pose Landmarker (heavy variant) [16]. The scale in centimetres per pixel is s = H / d(nose, left ankle), where d is the pixel distance between landmarks 0 and 27. This distance is shorter than the full stature, which excludes the crown of the head and the sole of the foot, so s, and therefore every width, is slightly overestimated.

3.2.3 Shoulder width from landmarks

The shoulder width is the distance between the two shoulder landmarks (11 and 12), $W_{shoulder} = s \cdot d(p11, p12)$. Unlike the hip landmarks, the shoulder landmarks lie close to the body surface, so the landmark distance is a reasonable width and is kept from the baseline.

3.2.4 Chest, hip and thigh widths from the silhouette

Let $w(y) = x_{max}(y) - x_{min}(y)$ be the horizontal extent of the pixels of the mask above a threshold of 0.5 in image row y. Widths are read at four heights: the chest at the mean shoulder height plus 0.05 of the image height, the hips at the mean height of the two hip landmarks (23 and 24), the thigh at the mean height of landmarks 23 and 25 plus 0.03 of the image height, and the waist at the height found as described next. The width in centimetres is $s \cdot w(y)$. The offsets 0.05 and 0.03 were chosen empirically and have not been calibrated against measured bodies. It should be noted that ISO 8559-1 defines clothing measurements such as the hip as girths measured at defined body levels [24], [25]; a silhouette width at the level of the hip landmarks is therefore a proxy, not the standard quantity.

3.2.5 Waist: hybrid detection with a plausibility check

No landmark marks the waist. Let T be the torso height, the vertical distance between the mean shoulder height and the mean hip height. The waist is searched in the window from 0.15 T below the shoulders to 0.10 T above the hips, which excludes the neck and collar region, and the row with the smallest width w(y) is taken. Its relative position in the torso is $\rho = (y^* - y_{shoulder}) / T$. If $0.35 \leq \rho \leq 0.65$, the result is accepted with the method label “segmentation” and confidence “high”. Otherwise the narrowest point is considered implausible, typically because of loose garments, and the waist height is set to $y_{shoulder} + 0.55\ T$, the width is read at that height, and the result is labelled “proportional_fallback” with confidence “low”. The model therefore never returns a waist value silently: the method and the confidence accompany every result.

3.2.6 Body-shape classification

The three widths shoulder S, waist W and hip H (in centimetres) are classified by rules, evaluated in the following order. Let $D = S - H$ and $M = \min(S, H) - W$. Rectangle: $|D| < 10$ and $|W - H| < 10$. Hourglass: $|D| < 8$ and $M \geq 15$. Apple: $|W - H| < 10$ and $M < 15$. Pear: $H - S \geq 8$. Inverted triangle: $S - H \geq 8$. Any other case is classified as rectangle, and “unknown” is returned if a width is missing. The thresholds are inch-based rules converted to centimetres (3 in ≈ 8 cm, 4 in ≈ 10 cm) and are a documented starting point rather than a calibrated anatomical model. The five classes are the same as those expected by the recommendation engine (Section 3.3). An earlier version of the classifier returned four different classes (athletic, triangle, rectangle, hourglass) that had no counterpart for pear or apple in the recommendation engine, which silently removed the body-shape signal from the ranking; aligning the two taxonomies corrected this interface defect. The classes are coarser than those of FFIT, which is based on girths and nine shapes [24]. Because the same body can change shape class when the definition of the measurements changes [24], the label is treated as an approximate input to the ranking, not as a diagnosis.

3.2.7 Size assignment

The size is chosen from a chart of shoulder-width limits (S up to 40 cm, M up to 44 cm, L up to 48 cm, otherwise XL). The chart also lists hip limits, but the function tests only the shoulder width, so hip width currently has no influence on the size. The server then validates the size against the hip measurement in inches, for example by requiring hips above 40 inches for size XL and forcing XL when hips exceed 40 inches, and the shopper can edit the size in the interface with a selector from XS to XL (Figure 4.6). The chart values are placeholders and must be replaced by a brand chart before production use.

3.2.8 Outputs and fallbacks

The service returns the measurements, body shape, size, the waist method and confidence, normalised vertical positions of the shoulder, waist and hip lines, and an annotated copy of the photograph in which the skeleton and three horizontal reference lines labelled with their values in centimetres are drawn. If the service does not answer within 15 seconds or fails, the server asks a vision–language model (Qwen2.5-VL [17]) for the same measurements under a prompt that states the sizing rules,.

3.3 Model 2: Outfit Recommendation Engine

Figure 3.3 summarises the engine: steps 1 to 6 are run once offline, and steps 7 to 9 are run on every shopper request.

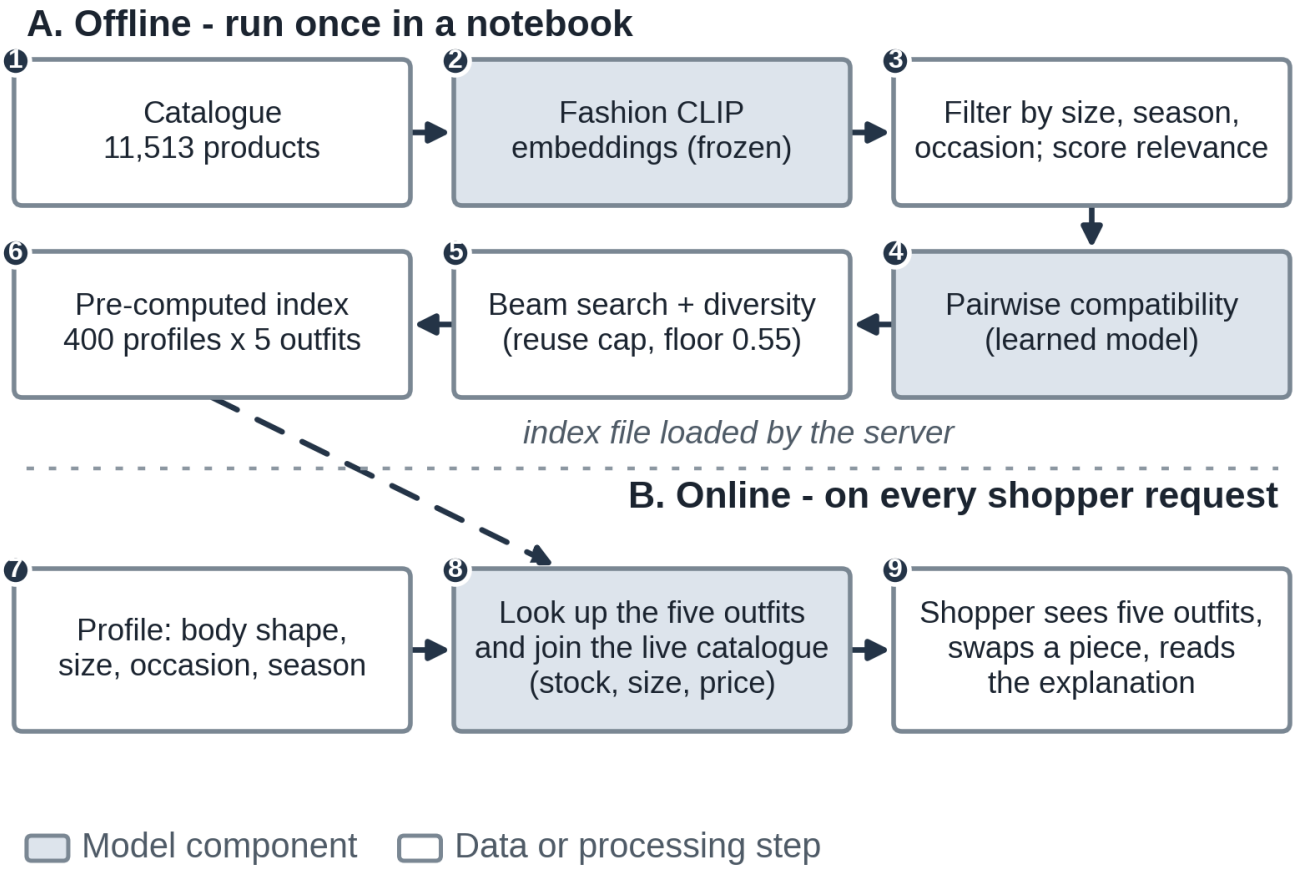


Figure 3.3. Pipeline of the outfit recommendation engine (A: offline preparation, B: online use).

3.3.1 Problem formulation

Let C be a catalogue of products, each with a garment category (Dress, Top, Bottom, Outerwear, Shoes or Accessory), a colour, a usage label, a season, the set of sizes in which it is stocked, and an image. A profile is a tuple (s, b, o, t, y) of clothing size, body shape, occasion, season and style. The task is to return K = 5 outfits, each

a set of products from C that forms a complete look (for example a dress with shoes and an accessory, or a top with a bottom), such that every product is available in size s, the outfits are well matched to (b, o, t, y), the garments within each outfit are mutually compatible, and the five outfits differ from one another.

3.3.2 Item representation

Each product is represented by a frozen Marqo-FashionCLIP embedding [3]. An image embedding v and a text embedding of the product description are computed, and the item embedding e is a weighted combination of the two (weights 0.7 for the image and 0.3 for the text). The image embedding is also kept separately and is the one used for body-shape scoring. Text prompts (occasion, silhouette, style) are embedded with the same model, so that cosine similarity between a product and a prompt is meaningful. The encoder is never fine-tuned.

3.3.3 Eligibility filtering

Products are first filtered so that only items stocked in the shopper's size remain. Season and usage are then matched to the profile. Because the dataset's usage field has no values for "wedding" or "interview", these occasions are mapped to the nearest usage values (formal and party for wedding; formal and smart casual for interview). Size is a hard constraint that is never relaxed. When strict filtering leaves too few candidates, the filter is relaxed in two stages, first ignoring season and then broadening usage, as described in Section 3.3.8.

3.3.4 Profile relevance

Each eligible product i receives three relevance signals, which are averaged with equal weights into a relevance score r(i) = (s_style(i) + s_body(i) + s_occ(i)) / 3.

- **Style:** s_style(i) is the cosine similarity between the item embedding and the embedding of a prompt of the form "A {style} {season} outfit".
- **Body shape:** s_body(i) = max over preferred prompts of cos(v_i, p) − max over avoided prompts of cos(v_i, q), computed on the image embedding. The preferred and avoided silhouette prompts for each of the five body shapes are listed in Appendix E.
- **Occasion:** s_occ(i) = cos(e_i, g) − λ · max(0, cos(e_i, g') − cos(e_i, g)), where g and g' are the embeddings of a descriptive "vibe" prompt and a contrasting "anti-vibe" prompt for the occasion, and λ = 2.0. A product is therefore penalised only when it is closer to the contrasting prompt than to the descriptive one.

For each garment category the best-scoring products form a candidate pool (Appendix B).

3.3.5 Pairwise compatibility learning

Compatibility between two products a and b is learned from pair features consisting of the absolute difference |e_a − e_b|, the element-wise product e_a ⊙ e_b, a six-dimensional category indicator for the pair, and a scalar colour-relationship score. Labels are catalogue-derived proxy labels, because no human-curated outfit corpus was available (Section 1.7). Pairs were built at product level so that no product appears in more than one of the training, validation and test splits (70%, 15%, 15%).

Two models are trained on these features. **CompatibilityNet v2** is a dual-path network. A visual branch (reduced in size, with stronger dropout) processes the embedding features; a metadata branch (widened, with weaker dropout) processes category and colour; and a fusion head receives both learned representations together with

the raw metadata vector through a skip connection. The revision from v1 was motivated by an ablation showing that the original visual branch (large relative to the number of training pairs) dominated a weaker metadata branch, the situation described as branch imbalance in [10]. Training uses the Adam optimiser with early stopping on the validation loss (settings in Appendix B). A **logistic-regression model** is trained on the same features. The two are combined post hoc as p = w · p_net + (1 − w) · p_lin, where w is chosen from a grid between 0 and 1 to maximise accuracy on the validation split only, and the chosen value is then applied unchanged to the test split. This is a simplification of the jointly trained wide-and-deep design of [9], which keeps the network architecture fixed while allowing the blend to shift towards the network if more labelled data becomes available.

3.3.6 Outfit construction, scoring and selection

Outfits are assembled from category templates (such as Dress, or Top with Bottom) extended with optional slots (such as Shoes and Accessory). Exhaustive enumeration of the Cartesian product of the candidate pools reaches tens of thousands of combinations for a single profile, so a beam search is used instead: at each step one category is added to every partial outfit and only the best 30 partial outfits, ranked by the same outfit-scoring function, are retained. The score of an outfit is final = 0.6 · compatibility + 0.4 · relevance − 0.1 · colour penalty, where the compatibility term pools the learned pairwise compatibility and occasion-consistency statistics of the outfit, relevance is the mean r(i) of its products, and the colour penalty applies to clashing non-neutral colours. Two safeguards follow. A **compatibility floor** removes any outfit whose weakest pairwise link falls below 0.55, so that one poorly matched pair cannot be hidden by well-matched ones. A **diversity selection** then walks the ranked outfits and accepts an outfit only if it is sufficiently dissimilar from those already chosen and does not use any product that already appears in two accepted outfits (reuse cap of 2). The cap plays a role analogous to the repetition-aware ranking of [7] and to diversity-promoting decoding [11].

3.3.7 Explanation generation

Each outfit is accompanied by a short justification. A language model is given only a structured evidence record (garment categories, colours, colour relationship, occasion, season, size, and whether the outfit was chosen by the model or customised by the shopper) and is instructed not to invent fabric, fit or price, not to quote raw scores, and not to use certainty language. If the model is unavailable, a deterministic template built from the same evidence is used. The ranking itself never depends on the language model.

3.3.8 Offline pre-computation of recommendations

Because the neural pipeline requires a GPU and the application should stay within free-tier limits, the engine is run offline for every combination of 4 sizes, 5 body shapes, 5 occasions and 4 seasons (400 profiles). For each profile a three-level fallback is applied: level 0 uses the strict filter; level 1 ignores season; level 2 additionally broadens the usage filter. Outfits found at later levels are added only if they do not duplicate earlier ones, until five outfits are obtained. For each profile the export stores the outfits, their deterministic explanations and evidence records, and a ranked candidate pool for each garment category. Before the export, the sizes held in the notebook are overwritten by the sizes actually stored in the shop's database, so that every recommended product is genuinely available.

3.4 Model 3: Virtual Try-On

Model 3 renders the garments of the selected outfit on the shopper's own photograph. It is not trained in this project: the pre-trained LaDI-VTON model with Dress Code weights [20], [26] is used as it is. The contribution lies in the pipeline around it, which turns ordinary shop and shopper photographs into the input that the model expects, applies the model to complete outfits, restores the parts of the person that must not change, and offers the model to the website as a service (Figure 3.4).

3.4.1 Design rationale and overview

LaDI-VTON expects its inputs in the Dress Code format: a 768 × 1024 photograph of the person, a 768 × 1024 photograph of the garment on a white background, a garment mask, a map of body-part labels and 18 pose keypoints in a 384 × 512 coordinate space. A shop catalogue provides none of these, and its garment photographs often show the garment on a mannequin or on a person. The pipeline therefore adds three stages around the model: garment cleaning (Section 3.4.2), person preparation (Section 3.4.3) and identity restoration after generation (Section 3.4.5). The model itself is run through the inference script of its official repository [28], with the Stable Diffusion 2 inpainting weights [27] loaded from a mirror on Hugging Face, because the original repository of those weights is no longer available (Appendix G).

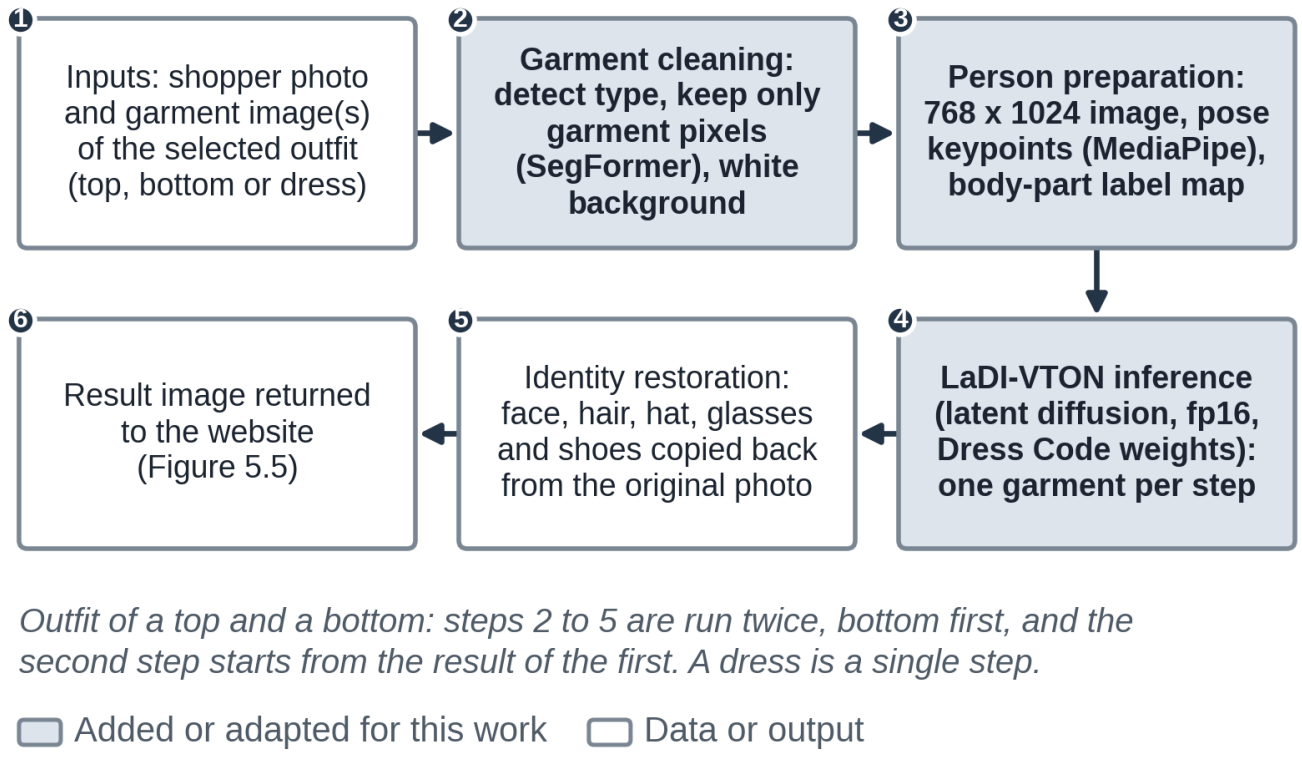


Figure 3.4. Pipeline of the virtual try-on model (Model 3) and the stages added around LaDI-VTON.

3.4.2 Garment cleaning

The garment photograph is first analysed by a human-parsing model (segformer_b2_clothes), which assigns every pixel to a class such as upper clothes, skirt, pants, dress or belt. Only the pixels that belong to the garment of the

requested category are kept; the mannequin, the person who wears the garment and the background are painted white. If the parser cannot find the garment, a background-removal mask (rembg) is used instead and a warning is printed. Transparent images are flattened onto white, and very small photographs, such as the 60 × 80 pixel images of the public catalogue, are enlarged before segmentation and flagged as low resolution, because the result will be blurred.

The garment type is detected from the parser's pixel counts (upper_body, lower_body or dresses); an ambiguous photograph, for example one that shows a full outfit, produces a warning. The website can also pass the category of the product explicitly. The catalogue categories are mapped to the three categories of the model: tops, shirts, jackets, sweaters and coats to upper_body; skirts, trousers, jeans and shorts to lower_body; and dresses to dresses.

3.4.3 Person preparation

The shopper's photograph is fitted into a 768 × 1024 canvas with white borders so that its proportions are kept. Pose keypoints are obtained with the MediaPipe pose landmarker [22] and converted to the 18-keypoint layout used by Dress Code, with the neck placed at the midpoint of the shoulders; landmarks whose visibility is below 0.3 are discarded. The body-part label map is produced by the same human-parsing model, and its labels are merged into the Dress Code identifiers: for the lower_body category, skirt pixels are merged into pants, and for the dresses category, upper-clothes, skirt, pants and belt pixels are merged into dress. The test photograph was a front-facing, full-body photograph of a person standing against a plain wall with the shoes visible.

3.4.4 Generation and layering of outfits

Each garment is processed by the inference script of LaDI-VTON for its category, in the unpaired setting in which the garment is not the one that the person is wearing, with mixed (fp16) precision and a batch size of one. The model changes one clothing region at a time, so an outfit made of a top and a bottom cannot be generated in a single pass. The garments are therefore processed in sequence, the bottom first and the top second, and the result of the first step becomes the person image of the second step, so that the top is layered over the bottom. A dress covers both regions: when a dress is present the other garments are ignored and a single step is run. Shoes, bags and other accessories are not supported by the model; they are ignored with a warning, and the interface lists them as part of the outfit without rendering them.

3.4.5 Identity restoration and delivery

Diffusion inpainting can alter regions that should remain unchanged. After each step, the pixels that the label map assigns to the hat, hair, sunglasses, face and shoes are copied back from the prepared original photograph through a mask that is blurred slightly (a Gaussian blur of radius 2) to avoid visible seams. The shopper therefore remains recognisable and the footwear is not regenerated. Because a generation takes minutes, the pipeline is wrapped in a small web service with a job queue, which the website submits to and polls (Section 4.5).

3.5 Evaluation Methodology

The compatibility models are evaluated on held-out product-level test pairs with accuracy, precision, recall, F1, ROC-AUC, PR-AUC, Spearman correlation and mean absolute error. They are compared with three non-learned baselines (a proxy compatibility formula, raw embedding cosine similarity and a category-only shortcut) and with a feature ablation that isolates the visual signal, the metadata signal and their combination. Outfit-set diversity is

measured by the average pairwise outfit similarity and the number of unique items in the top five outfits before and after the reuse cap. The integrated pipeline is evaluated end to end on several profiles. Ranking stability is measured as the overlap between the top-five outfit sets obtained under different relevance/compatibility weightings, in the spirit of weight-stability analysis [12]. The results are reported in Section 5.

The body-measurement model was examined through case studies in a notebook: each step of the pipeline was run on test photographs of people wearing different garments, the intermediate values (landmark distances, silhouette widths, the position of the narrowest point of the torso) were inspected, and the behaviour of the baseline and of the corrected method was compared (Section 5.2). No tape-measure ground truth was collected, so the case studies diagnose behaviour but do not estimate accuracy.

3.6 Development Approach, Tools and Technology Choices

The platform was developed iteratively by a team of four students, with the source code kept under version control on GitHub. The AI components were prototyped and evaluated in notebooks before being integrated, and each integration step was checked by compiling the front end and server with the TypeScript compiler. Table 3.1 lists the main tools and the reasons for choosing them. The dominant constraint was cost: every managed service was selected for a free tier that is sufficient for a project of this size.

Table 3.1. Tools and technology choices.

Concern	Choice	Reason
Front end	React with Vite and Tailwind CSS	Component-based interface and fast development cycle
Back end	Node.js with Express and TypeScript	One language across client and server; simple REST APIs
Authentication	Amazon Cognito	Managed user pool and signed tokens that the server can verify; no password handling in the application
Database	Amazon DynamoDB	Managed key-value store with a free tier; low operational effort
Infrastructure	Terraform	Cloud resources are described in code and can be recreated
Media storage	Supabase Storage	Free public object storage for images and product thumbnails
Body measurement	Python with MediaPipe Pose Landmarker, OpenCV, NumPy and Flask	Off-the-shelf pose and segmentation model [22] that runs on a CPU and needs no training data; a small local service keeps it separate from the Node.js server
Recommendation model	Python notebook on a free GPU runtime, frozen fashion CLIP embeddings	Neural inference is needed only offline; the web server then performs a file look-up (Section 3.3.8)
Virtual try-on	LaDI-VTON (PyTorch, diffusers) with Dress Code weights on a free Colab T4 GPU; FastAPI job service; SegFormer parsing and MediaPipe keypoints	Open code and weights, three garment categories and no per-call quota; the GPU is needed only for this model, so it is run as a separate service (Section 4.5)
Explanations	Large language model API (free tier)	Natural-language justification without training a text model; a template is used as fallback

4. Design and Implementation

4.1 Architecture Overview

FitStyle AI follows a three-tier design. A React single-page application provides the shopper interface; a Node.js/Express server provides authenticated APIs and orchestrates the AI components; and managed cloud services store identity and data; the client, the server and the measurement service run locally, whereas the database, authentication, image storage, language model and try-on service are cloud or external services. Figure 4.1 shows how the components are connected and where they run, and Table 4.1 summarises the layers.

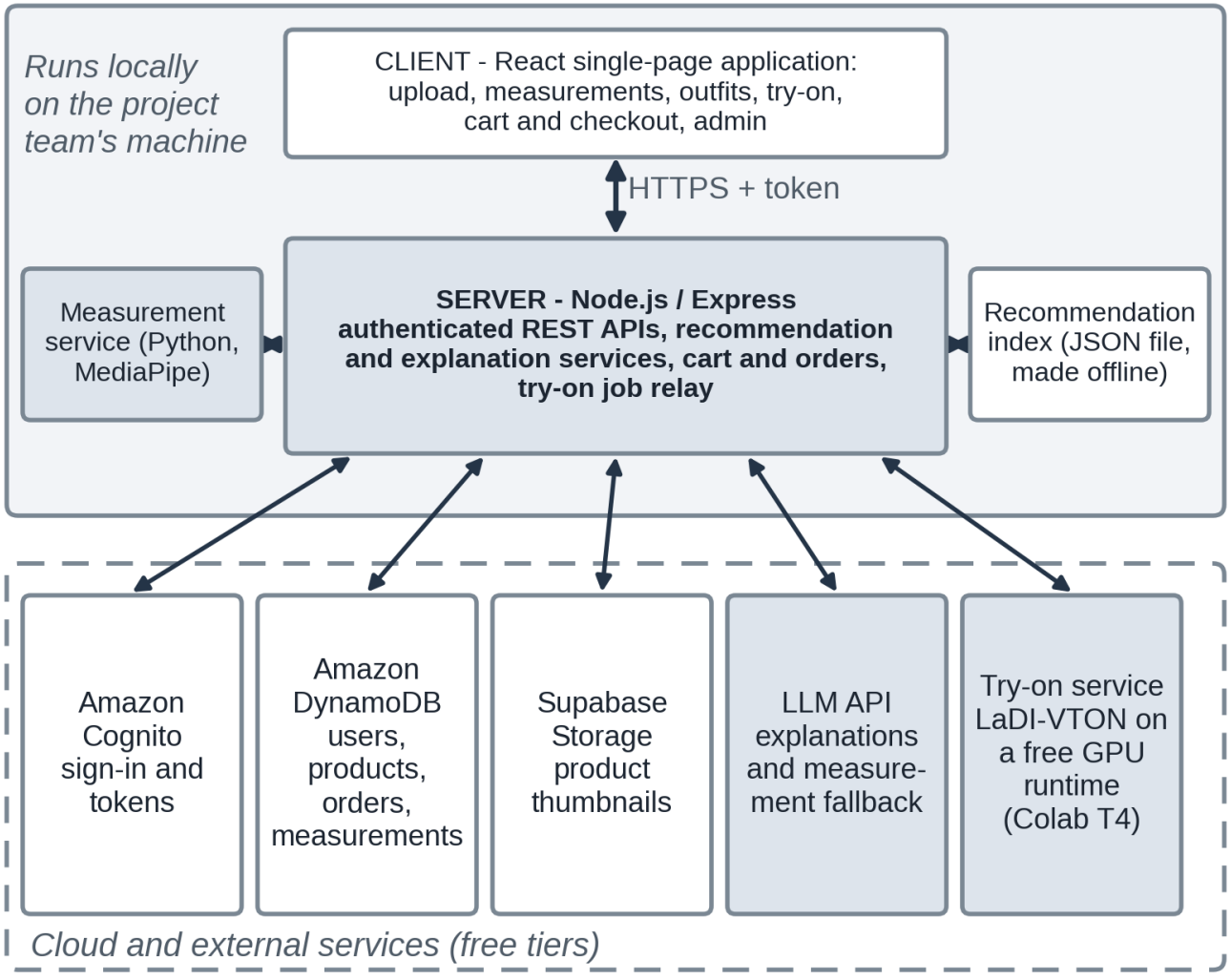


Figure 4.1. System architecture of FitStyle AI.

Table 4.1. Layers of the FitStyle AI implementation.

Layer	Technology	Role
Client	React, Vite, Tailwind CSS	Photo upload, measurement review, outfit selection, virtual try-on, cart and checkout

Layer	Technology	Role
Server	Node.js, Express	Authenticated REST APIs; joins recommendations with the catalogue; calls external AI services
Body measurement	Python (Flask), MediaPipe Pose Landmarker, OpenCV	Local micro-service that returns widths, body shape, size and an annotated photograph
Virtual try-on	Python, LaDI-VTON, FastAPI job queue, Google Colab T4 GPU, temporary public tunnel	GPU service that accepts the shopper's photograph and garment image URLs and returns the try-on image after a few minutes
Identity	Amazon Cognito	Sign-up, sign-in and token-based authentication of every API request
Data	Amazon DynamoDB (Users, Products, Orders, Measurements)	Catalogue, profiles, carts, orders and measurements
Infrastructure as code	Terraform	Reproducible provisioning of cloud resources
Media	Supabase Storage (public bucket)	Product thumbnails and uploaded images
Recommendation index	JSON file loaded at server start	Pre-computed outfits and candidate pools for the 400 profiles
Explanations	Large language model API	Natural-language justification of the selected outfit

4.2 Data Design

The product catalogue is stored in DynamoDB and loaded into server memory at start-up, so that price, stock, available sizes and colour are always taken from the database rather than from the recommendation index. Each product carries a rich six-value garment category that the migration script stored alongside the coarser four-value category used by older parts of the interface. Measurements accepted by the shopper are stored in the Measurements table. The cart is stored with the user's record and is emptied by the server after a successful payment. The number of DynamoDB tables and their provisioned capacities were deliberately left unchanged, because the existing four tables already use the available free-tier read capacity; this constraint is the main reason recommendations are pre-computed instead of stored in a new table.

4.3 Recommendation Service

The recommendation service is a server module that exposes three authenticated endpoints (Appendix A). The index file maps a profile key (size, body shape, occasion, season) to up to five outfits, each with its product identifiers, garment categories, score, deterministic explanation and evidence, together with ranked candidate pools per category. When a request arrives, the service (i) normalises the profile (mapping XS to S and XXL to XL because the catalogue stocks only S to XL, and defaulting the season from the current date), (ii) looks up the profile, (iii) joins each identifier with the live catalogue, (iv) discards pieces that are out of stock or no longer offered in the shopper's size, and (v) returns each piece displayed in the shopper's size. If the index is missing or the call fails, the interface silently falls back to the earlier collection list, so the recommendation layer can be absent without breaking the application.

The explanation endpoint accepts the identifiers of the pieces currently selected rather than an outfit identifier, so that the justification always describes exactly what the shopper has chosen. Requests are limited by a 12-second timeout, results are cached in memory by profile and selection, and the deterministic template is returned if the language model fails.

4.4 User Interface

In the try-on step the left panel shows the recommended outfit, with the outfit title, its total price and, for each piece, the image, name, price, colour and size. A control opens all five outfits, from which the shopper selects one; the selection panel on the right then shows its pieces. Selecting a piece opens a dialog in the centre of the screen listing the other suitable products of the same garment category in the shopper's size; choosing one replaces the previous piece. Removing a piece from the selection panel un-selects it but leaves it in the outfit on the left, where it can be re-selected. A card below the preview explains why the outfit was chosen and is regenerated whenever the selection changes; it is labelled "Customised" when the outfit differs from the model's pick. Selected pieces are added to the cart in the shopper's size (Figure 4.2).

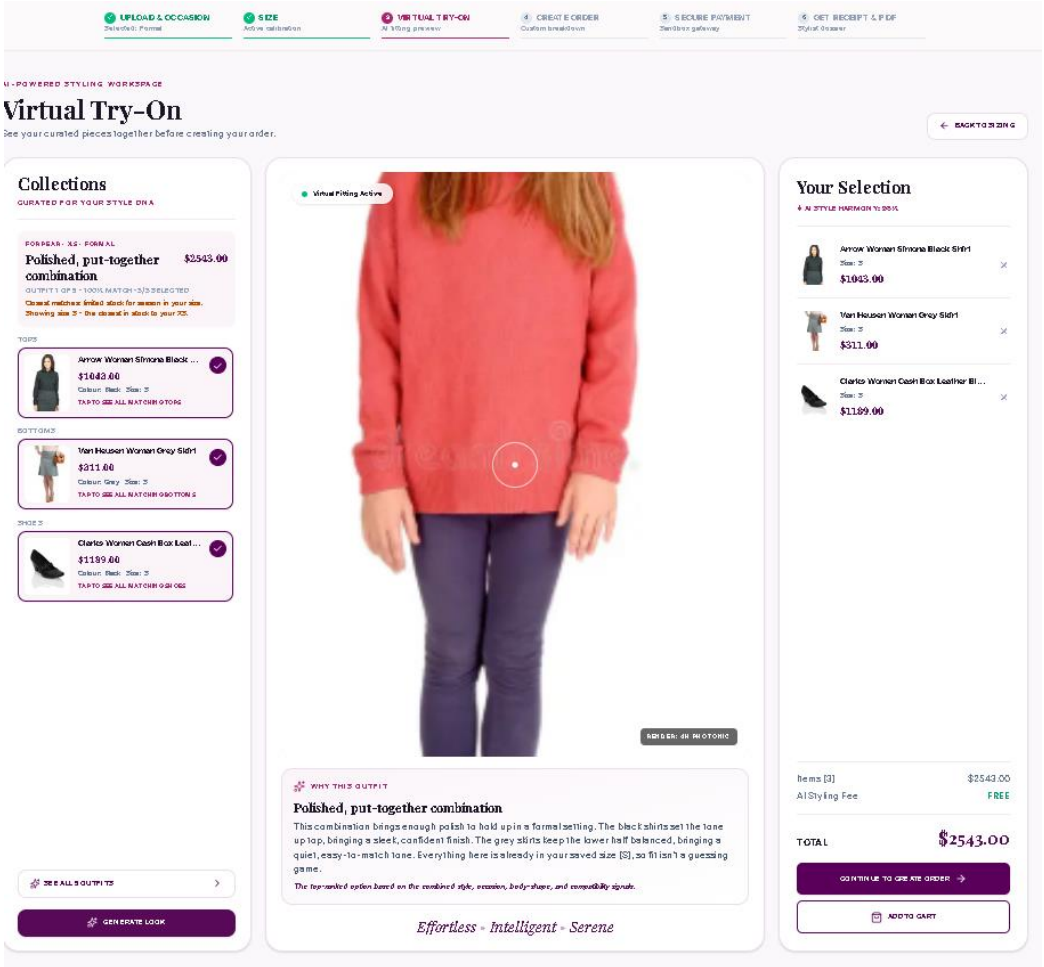
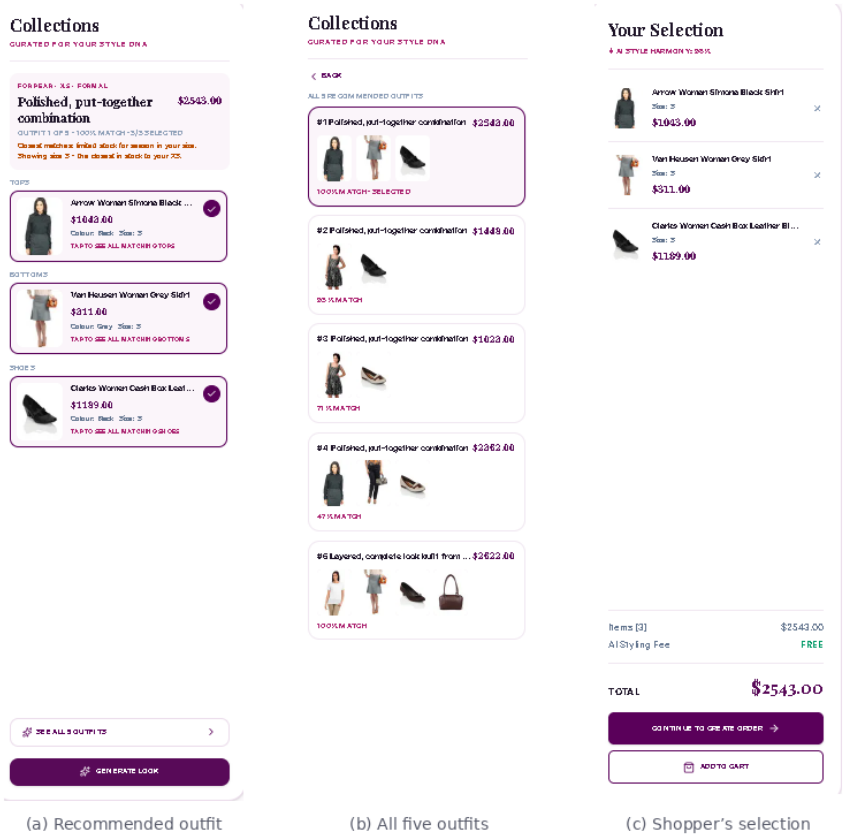


Figure 4.2. Try-on step: the recommended outfit (left), the preview of the shopper (centre) and the shopper's selection (right).

The left panel and the selection panel are shown in more detail in Figure 4.3: (a) the recommended outfit with each piece grouped by category, (b) the list of all five recommended outfits and (c) the shopper's selection with its total and the Add to Cart and Continue buttons.



(a) Recommended outfit (b) All five outfits (c) Shopper's selection

Figure 4.3. Panels of the try-on step: (a) recommended outfit, (b) all five outfits and (c) shopper's selection.

Selecting a piece opens a dialog that lists the other suitable products of the same category in the shopper's size (Figure 4.4). The explanation card (Figure 4.5) is regenerated whenever the selection changes.

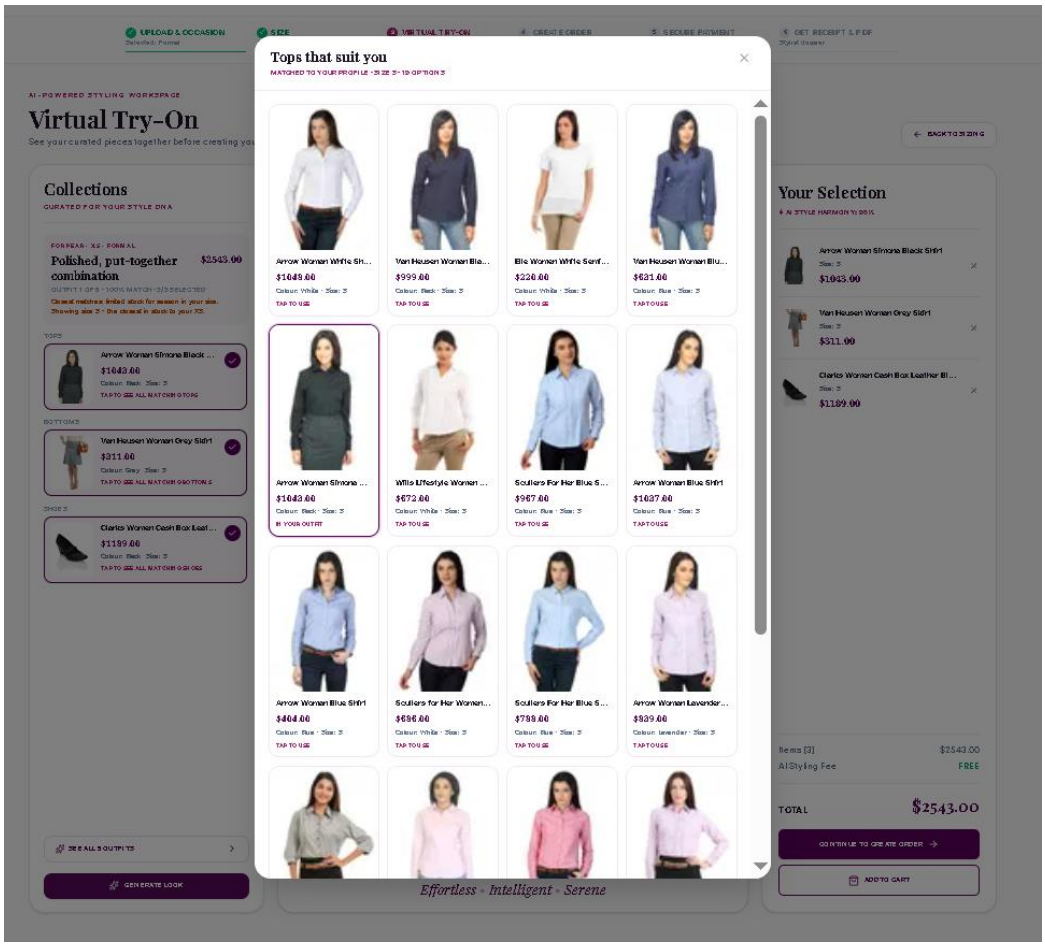


Figure 4.4. Dialog listing the tops that suit the shopper's profile; choosing one replaces the current top.

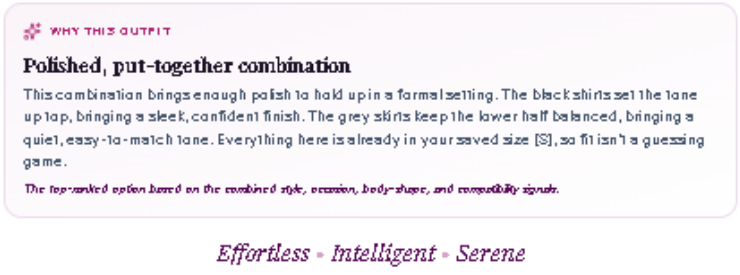


Figure 4.5. Explanation card ("Why this outfit") shown below the preview.

4.5 Integration of the AI Models

Body-measurement service. Model 1 is implemented as a Python micro-service (Flask with cross-origin access enabled) that listens on port 5001 and exposes two endpoints: a POST endpoint that accepts the photograph as a base64 string and the height, and a health endpoint. The service writes the image to a temporary file, deletes it after processing, and answers with HTTP 422 if no person is detected and HTTP 500 for other failures. It loads the MediaPipe Pose Landmarker heavy model (about 30 MB), which it downloads on first start, with segmentation output enabled. The functions of the notebook (waist detection, shape classification, size mapping and the main

detect_size routine) were moved unchanged into the service file, with the notebook-only display code removed, so that the experiments in Section 5.2 describe the deployed behaviour.

The Express server orchestrates the call in its body-analysis endpoint, applies the validation rules and the fallbacks of Section 3.2.8, and stores the accepted measurements with the user's profile. In the browser, the measurements are shown as editable cards beside the photograph, and the normalised line positions returned by the service allow the interface to draw its own live reference lines whose labels follow the values the shopper edits (Figure 4.6).

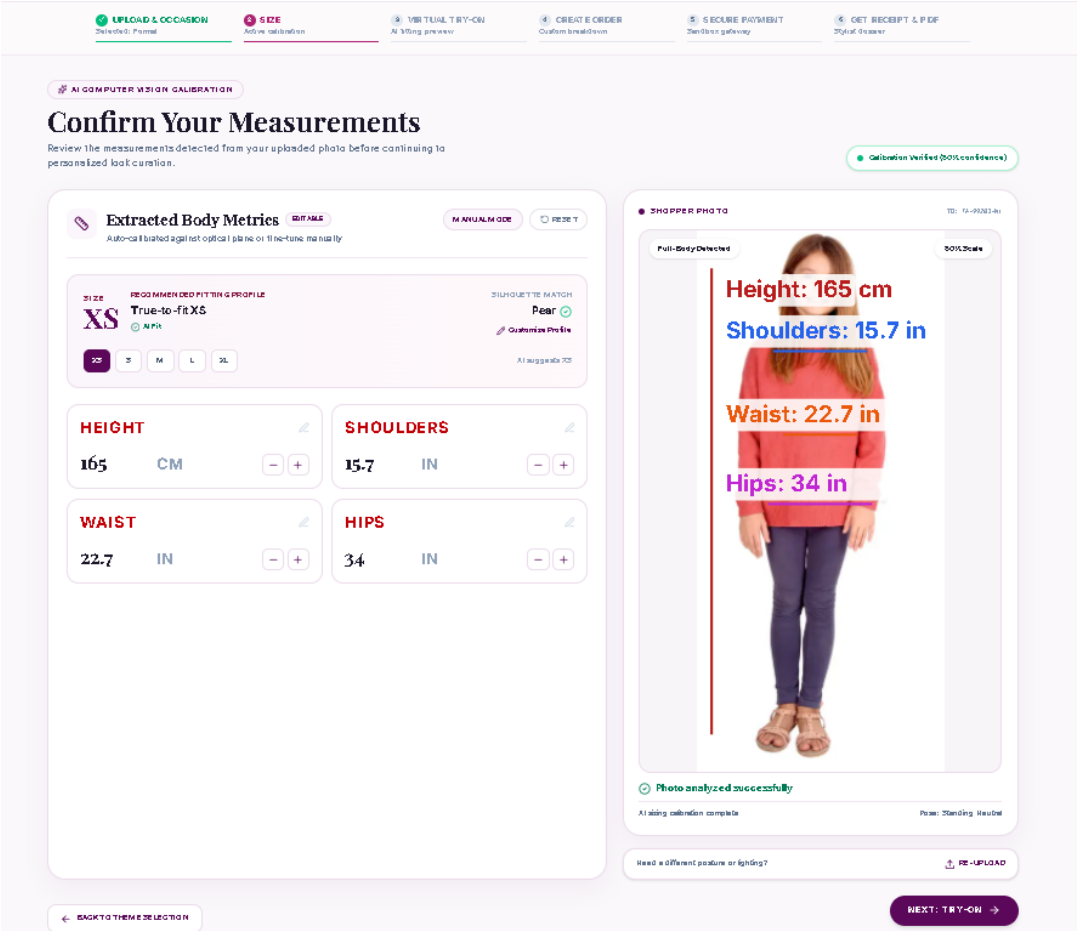


Figure 4.6. Body-measurement step: extracted metrics with an editable size selector (left) and the annotated photograph (right).

Virtual try-on service. Model 3 needs a GPU, which neither the team's machines nor the free tiers of the cloud services provide, so it runs as a separate service in a notebook on a free Colab T4 GPU. The notebook starts a FastAPI application on port 8000 and exposes it through a temporary public address created with cloudflared; the address and an API key are printed when the cell is run and change each time. A POST request to /tryon uploads the shopper's photograph (limit 10 MB) together with a JSON list of one to three garments, each with the URL of its product image and, optionally, its category, and answers at once with a job identifier. A single worker thread takes the jobs from a queue and runs the pipeline of Section 3.4, because one GPU can serve only one generation at a time. The site polls GET /jobs/{id} for the status (queued, running, done or error) and, when the job is done, downloads the PNG image from GET /results/{id}. The API key is held by the Express server, not by the browser,

and the uploaded photograph is deleted as soon as the job finishes; results are deleted after one hour (Appendix K).

In the browser, the shopper presses Generate Look. The centre panel then shows the message “Generating try-on… (may take a few minutes)” over the photograph, and it replaces the photograph with the generated image, labelled “Virtual Fitting Active”, when the job is done (Figures 5.5 and H.11).

The interface from Model 1 to Model 2 consists of the body-shape label and the clothing size. The interface from Model 2 to Model 3 is the image of the main garment of the selected outfit (the dress if present, otherwise the top). The service itself accepts up to three garments per job and can layer a top over a bottom (Section 3.4.4).

4.6 Security and Cost Considerations

Every recommendation endpoint requires a valid Cognito token, and the cart and checkout endpoints additionally verify that the token belongs to the owner of the cart. The language-model key is never exposed to the client, and all calls to external services are made by the server. The recommendation design avoids paid components: inference runs once offline on a free notebook runtime, thumbnails use free object storage, and the server performs only a file look-up and a join per request.

4.7 Implementation Challenges

Table 4.2. Integration challenges and the measures taken.

Challenge	Cause	Measure
Recommended products in sizes the shop does not stock	The notebook and the database had generated different random size lists	Overwrite the notebook sizes with those exported from the database before computing the index
Blank product images	The migrated catalogue contains no image links	Upload thumbnails for the recommended products to object storage and derive their URL from the product identifier
Fewer than five outfits for narrow profiles	Strict filtering on size, season and usage leaves few candidates; wedding and interview match no usage values	Three-level fallback with a usage mapping; size is never relaxed
Measurement service aborted on some photographs	MediaPipe aborts the process when the mask width is not a multiple of four pixels	Pad the right edge of the image by replicating the border
Body-shape class unknown to the recommendation engine	The measurement module returned four classes and the engine expects five	Align the taxonomy and the thresholds
Shopper size XS	The catalogue stocks only S to XL	Map XS to S and tell the shopper that the closest size is shown
Dresses and outerwear collapsed into “top”	The interface used a four-value category	Carry the six-value garment category through the API and the interface
Catalogue photographs unsuitable as try-on input	Many product photographs show the garment on a mannequin or a person, and the public catalogue images are only 60 × 80 pixels	Garment cleaning with human parsing and a white background; enlargement and a low-resolution warning (Section 3.4.2)

Challenge	Cause	Measure
Model weights no longer available	The Stable Diffusion 2 inpainting weights named in the model's code were removed from their original repository	Point the model code to a mirror of the same weights (Appendix G)
Library conflicts	The model needs older versions of PyTorch, diffusers and transformers than the notebook runtime provides	A separate Python 3.10 environment with pinned versions (Appendix G)
Generation takes minutes on an external GPU	Diffusion inference on a T4 GPU is slow, and a notebook runtime disconnects and gives a temporary public address	Job queue with polling; photographs are deleted after use; the set-up is documented as suitable for development and demonstrations (Appendix K)

4.8 Platform Features

The application covers the complete journey of a shopper and the management of the shop. Table 4.3 lists its features, and the corresponding screens are reproduced in Appendix H and in Section 4.4. Checkout is simulated in test mode, so no real money is taken, and a pay-in-store option reserves the order for payment at pickup. Store-owner accounts are created by invitation only. Cart and checkout requests are authenticated and restricted to the owner of the cart (Section 4.6).

Table 4.3. Features of the application.

Area	Feature	Screen
Access	Landing page that introduces the application and leads to sign-in	Fig. H.1
Access	Sign-up and sign-in for shoppers with e-mail and password or with Google; separate sign-in for store owners; store-owner sign-up by invitation only	Fig. H.2
Shopper	Step 1: upload a full-body photograph and choose an occasion (Wedding, Formal, Casual or Party)	Fig. H.3
Shopper	Step 2: review the measurements, body shape and size inferred from the photograph; edit them manually, reset them, choose another size or upload another photograph	Fig. 4.6
Shopper	Step 3: five recommended outfits with match score and total price; view all five and choose one	Figs. 4.2, 4.3
Shopper	Swap any piece for another suitable product of the same category in the shopper's size; select or deselect pieces	Fig. 4.4
Shopper	Explanation of the selected outfit, generated by a language model and refreshed when the selection changes	Fig. 4.5
Shopper	Virtual try-on preview of the selected garment on the shopper's photograph (Generate Look)	Figs. 4.2, 5.5, H.11
Shopper	Add the selection to the cart; change quantities, remove items, enter a promo code and choose to buy online or pay offline	Fig. H.5
Shopper	Step 4: create the order from the chosen pieces and export a PDF summary	Fig. H.4
Shopper	Step 5: payment in a sandbox gateway with a test card (no real currency is debited)	Fig. H.6

Area	Feature	Screen
Shopper	Step 6: order confirmation with delivery tracking and a downloadable PDF summary	Fig. H.7
Shopper	Pay offline: the order is reserved, and the PDF summary carries a QR code and the shop location for payment at pickup	Fig. H.8
Store owner	Inventory dashboard: catalogue summary, search, filters by category, occasion and size, sorting, stock switch, edit and delete, add garment and export to CSV	Fig. H.9
Store owner	Sales and analytics dashboard: revenue, sales by occasion, turnover by category, top-selling products and recent customer orders	Fig. H.10

Source: the implemented application; the screens are shown in Appendix H and in Figures 4.2 to 4.6.

5. Results and Evaluation

5.1 Evaluation Protocol and Data

All recommendation experiments use the catalogue of 11,513 products described in Appendix D, drawn from a public fashion-product dataset [14]. Products were split at the product level (70% training, 15% validation, 15% test), so that no product appears in more than one split, and the compatibility models were trained on 948 labelled product pairs. Compatibility labels are proxies derived from catalogue attributes, not human judgements, so every accuracy figure below measures agreement with that proxy. All figures come from the project notebook (version v20, fixed random seed) and refer to the held-out test split unless stated otherwise.

5.2 Body-Measurement and Size Model (Model 1)

Experimental setting. The module was examined in a notebook on a small number of test photographs of people wearing different garments. No tape-measure ground truth was collected. The results in this section are therefore case studies that diagnose the behaviour of the method; they are not accuracy estimates. Figure 4.6 shows the output of the module in the application.

Table 5.1. Hip-width estimate for one test photograph: landmark baseline and segmentation-based method.

Quantity	Landmark baseline	Segmentation-based method
What is measured	Distance between the two hip joint centres	Width of the silhouette at hip height
Hip estimate (cm)	19.5	61.9
Shoulder width (cm)	about 38	about 38

Source: FitVerse_Detect_Size_Module notebook, one test photograph. The shoulder width is measured in the same way by both methods.

The landmark baseline gave a hip estimate of about half the shoulder width. The cause is that it measures a different quantity: the hip landmarks are internal joint centres, whose separation is smaller than the width of the body at the hips. Reading the mask at hip height gave about three times that value. The new value must be interpreted with care: it is a straight-line silhouette width, not a circumference, and it is large relative to the shoulder width, because the silhouette at hip height can include arms or hands resting at the sides, loose clothing and any overestimate of the scale. Without tape-measure ground truth the correction is plausible but not validated.

Table 5.2. Behaviour of the hybrid waist detection across garment types.

Garment scenario	Relative position of the narrowest point	Fallback triggered	Observation
Loose cardigan with scarf or hijab	about 0.15	Yes	False narrow point at the neck and scarf; a wrong answer with apparent confidence
Fitted blouse worn untucked, with trousers	about 0.15	Yes	No local minimum: the width increases steadily because the blouse drapes past the waist
Fitted, tucked-in clothing	not yet tested	expected: no	Planned; the body contour should be visible to the mask

Source: notebook test photographs. Positions are fractions of the torso height; the plausible range used by the method is 0.35 to 0.65.

Two distinct failure modes were found. In the first, the segmentation produces a false narrow point; in the second, it produces no narrow point at all. Both fall outside the plausible range of the torso height and are caught by the plausibility check, which then applies the proportional estimate and reports low confidence. The check protects against silent errors but does not recover the true waist width in these cases. The third scenario, in which the contour of the body is directly visible, has not yet been tested.

5.3 Outfit Recommendation Engine (Model 2)

5.3.1 Compatibility models

Table 5.3 reports the held-out performance of the compatibility models. The rebalanced dual-path network (v2) improved on its first version and on a category-only shortcut, but the linear model trained on the same features was better still. Figure 5.1 shows that the metadata signal alone is much stronger than the visual signal alone, and that adding the visual signal to the metadata gave only a small gain.

Table 5.3. Held-out accuracy of the compatibility models.

Model	Accuracy	F1	ROC-AUC
Category-only shortcut	0.536	–	–
CompatibilityNet v1	0.646	–	–
CompatibilityNet v2	0.789	0.802	0.899
Linear model	0.932	0.934	0.980

Source: project notebook (version v20), held-out test split of 237 labelled pairs; the values for the first network version and the category-only shortcut are taken from the project's earlier report. A dash means that the value was not reported.

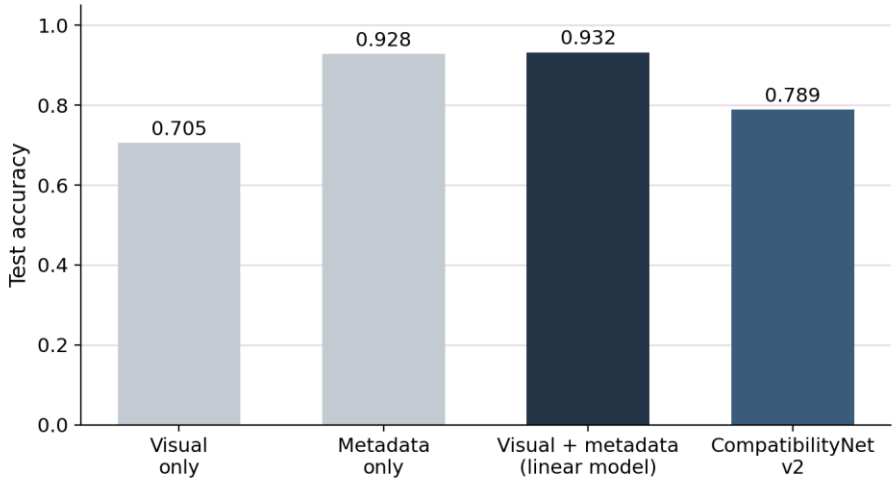


Figure 5.1. Feature ablation: test accuracy with visual features only, metadata only, both (linear model) and with the CompatibilityNet v2.

5.3.2 Ensemble with a validation-selected weight

A logistic-regression model trained on the same features was blended with the network as score = w · network + (1 − w) · linear, with w chosen on the validation split only. Validation accuracy fell as more weight was given to the network (Figure 5.2), and the selected weight was 0, so on this data the ensemble reduces to the linear model.

The test accuracy of 0.932 is therefore the linear model's own performance and should not be attributed to the neural network. This was the outcome in every re-run, and the weight is selected again whenever the models are retrained. The result is a finding about the scale of the labelled data, consistent with the trade-off between memorisation and generalisation discussed in [9].

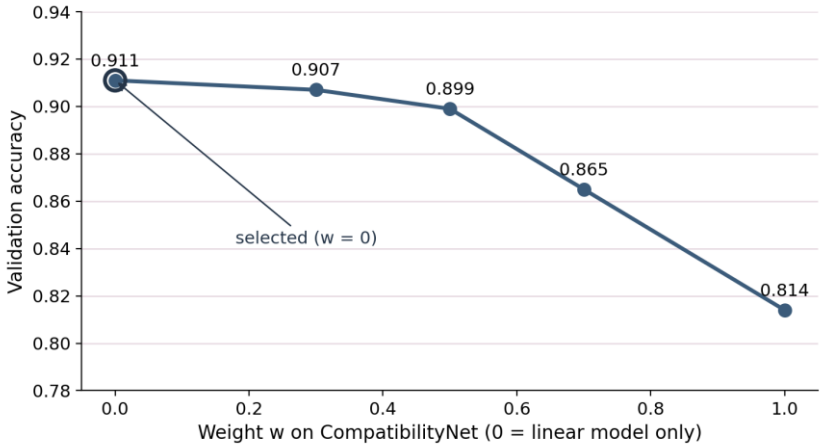


Figure 5.2. Validation accuracy for different weights on the CompatibilityNet (w = 0 is the linear model alone).

5.3.3 Diversity of the recommended set

A per-item reuse cap increased the number of distinct items across the top five outfits from 8 to 14, with a very small loss of relevance (Figure 5.3). The cap operates on product identifiers, so visually near-identical items (for example several grey handbags) can still recur under different identifiers; this is a recognised limitation.

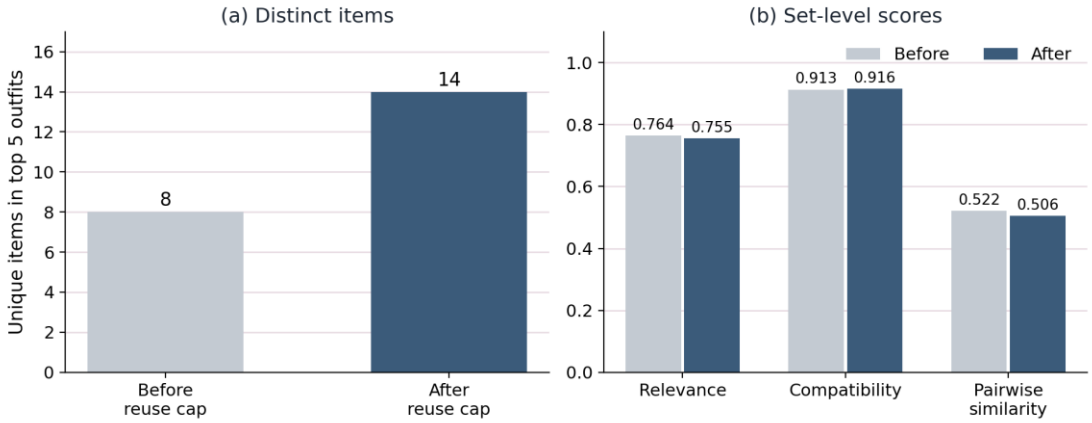


Figure 5.3. Effect of the per-item reuse cap on the number of distinct items (a) and on set-level scores (b).

5.3.4 End-to-end behaviour

To check the integrated pipeline, it was run for several profiles. For narrower profiles, such as a formal winter profile in size S, strict filtering on size, season and usage left too few eligible products, and fewer than five outfits satisfied the similarity threshold and the reuse cap. This limitation was not visible in component-level tests and motivated the fallback described in Section 5.3.6. Figure 5.4 shows the five outfits returned by the notebook for one profile.

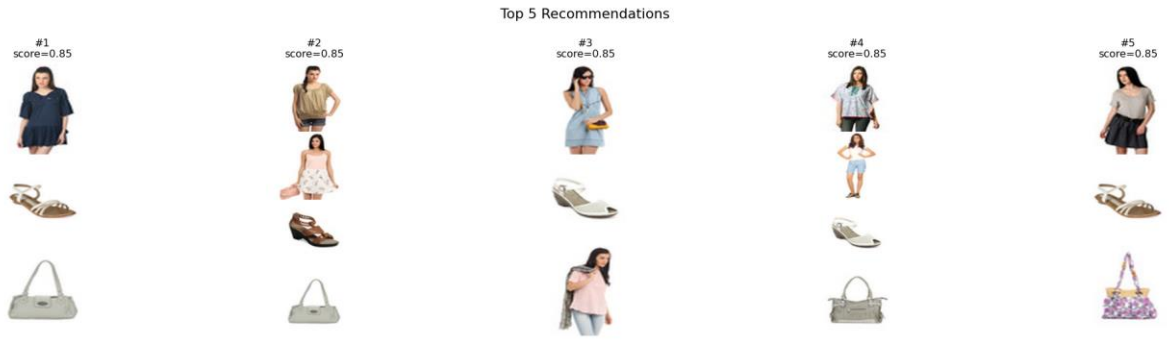


Figure 5.4. Top five outfits returned by the notebook for a sample profile.

5.3.5 Sensitivity to the weighting

Changing the balance between relevance and compatibility changed which outfits appeared in the top five, from almost complete overlap with the default set to none, depending on the weighting and the run. Following the idea of weight stability [12], this indicates that the default weighting (compatibility 0.6, relevance 0.4) lies in a narrow stability interval; it is therefore a stated design preference and not an empirically validated optimum.

5.3.6 Deployed system: pre-computed recommendations

To serve recommendations without running the neural pipeline on every request, the engine is run once offline for every combination of 4 sizes, 5 body shapes, 5 occasions and 4 seasons (400 profiles). To address the sparse-pool limitation, a three-level fallback was added: level 0 applies the strict filter, level 1 ignores the season, and level 2 also broadens the usage filter. Size is always enforced. The occasions wedding and interview, which have no matching value in the usage field of the dataset, are mapped to the nearest usage values. For each profile the export stores up to five outfits, their deterministic explanations and a ranked candidate pool for each garment category, which is used when the shopper replaces a single piece. At request time the server joins these identifiers with the live catalogue held in DynamoDB, removes any piece that is out of stock or no longer offered in the shopper's size, and shows each piece in the shopper's size.

The justification shown with each outfit is generated at request time by a language model that receives only structured evidence about the outfit (garment categories, colours, colour relationship, occasion, season and size) and is instructed not to invent attributes or to quote raw scores; a deterministic template is used if the model is unavailable. The explanation is regenerated whenever the selection changes and is marked as customised when the outfit differs from the model's pick.

5.4 Virtual Try-On Model (Model 3)

Model 3 was evaluated qualitatively. The pre-trained model was neither trained nor fine-tuned in this project, a shopper has no ground-truth photograph of herself wearing the product, and neither a reference-free image-quality metric nor a user study was run. This section therefore reports what was observed in the notebook runs on a Colab T4 GPU and in the website, and the limitations that follow.

Three kinds of input were used. A front-facing, full-body photograph of one person standing against a plain wall served as the shopper. The garment photographs came from the product images of the project (a belted shirt

dress) and from the public catalogue of 60 × 80 pixel photographs (a T-shirt and trousers). The website run used the products of a recommended outfit. Table 5.4 lists the runs and Figure 5.6 shows the notebook results.

Table 5.4. Try-on runs recorded in the notebook and in the website.

Run	Garment input	Detected category and warnings	Time on T4	Observation
Dress (notebook, Step 10)	Belted shirt dress, product image of the project	dresses; warning that the photograph is ambiguous between an upper-body garment and a dress	462 s	Collar, short sleeves and belt of the dress are rendered; face, hair and shoes are those of the original photograph
Two-piece outfit (notebook, Step 11)	T-shirt and trousers, 60 × 80 pixel catalogue images	lower_body, then upper_body; low-resolution warning for both	343 s + 90 s	Both garments applied, bottom first; the result is blurred and shows an artefact at the waist
Website run (Figure 5.5)	Top of the recommended outfit, a white blouse with a bow	Top, passed by the site	Not timed	Top rendered on the shopper; the lower body is unchanged because only the main garment is sent

Source: Model 3 notebook (steps 10 and 11) and the website run of Figure 5.5; times from single runs on a Colab T4 GPU.

In the one-piece run the dress was recognised although the parser labelled 109,108 pixels as upper clothes and 153,110 pixels as dress, and the generated image shows the dress with the collar, the short sleeves and the belt while the face, hair and shoes of the original photograph are kept. In the two-piece run both garments were detected and applied, the trousers first, but the catalogue images are so small that the garments are blurred, and the result shows an artefact at the waist. The notebook notes that the quality of the second step is slightly lower than that of the first, because it starts from an image that was itself generated. The times are those of single runs; the first run of a session also loads the model, so they indicate the order of magnitude, minutes per outfit, and not a benchmark.

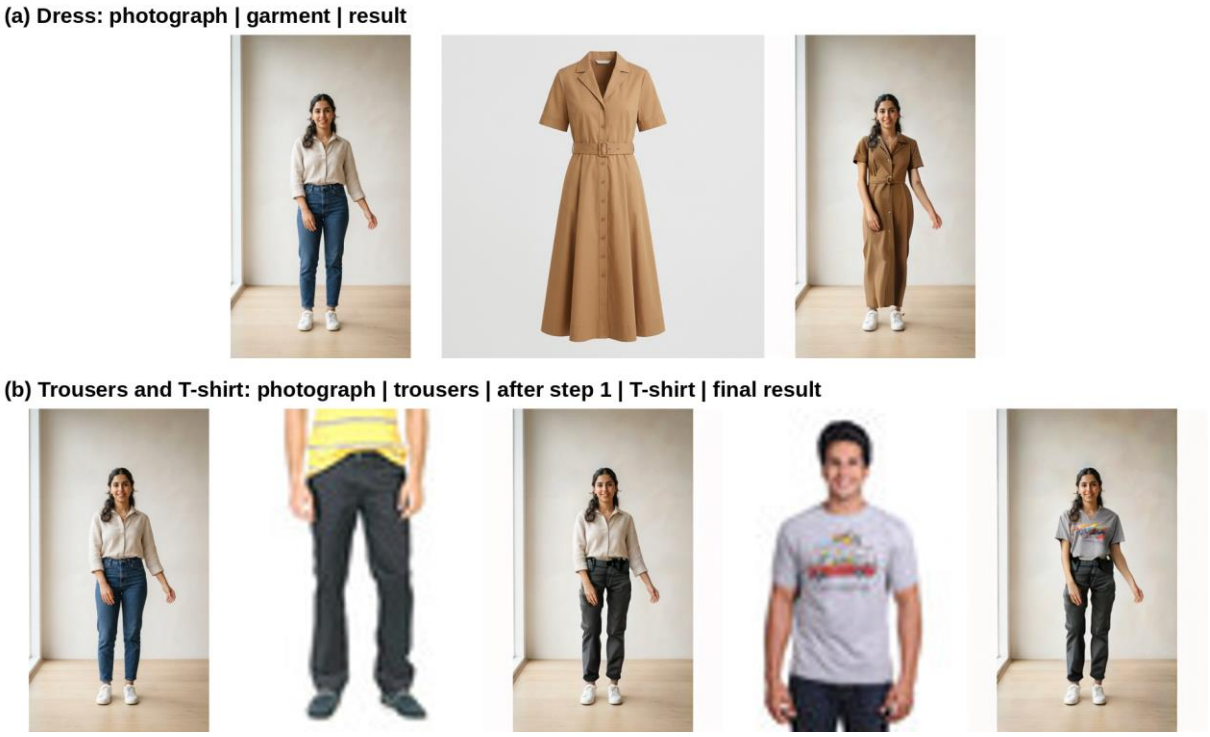


Figure 5.6. Try-on runs in the notebook: (a) a dress; (b) trousers and a T-shirt from 60 × 80 pixel catalogue images. Each row shows the shopper's photograph, the garment image and the result; in (b) the trousers are applied first and the T-shirt second.

Figure 5.5 shows the shopper's photograph before and after a generation requested from the website. The label of the preview changes from "Preview" to "Virtual Fitting Active". The top of the selected outfit is rendered on the shopper, while the trousers are still the shopper's own jeans, because the website passes only the main garment to the service (Section 4.5); the shoes and the handbag of the outfit appear only in the selection list, because the model cannot render them.

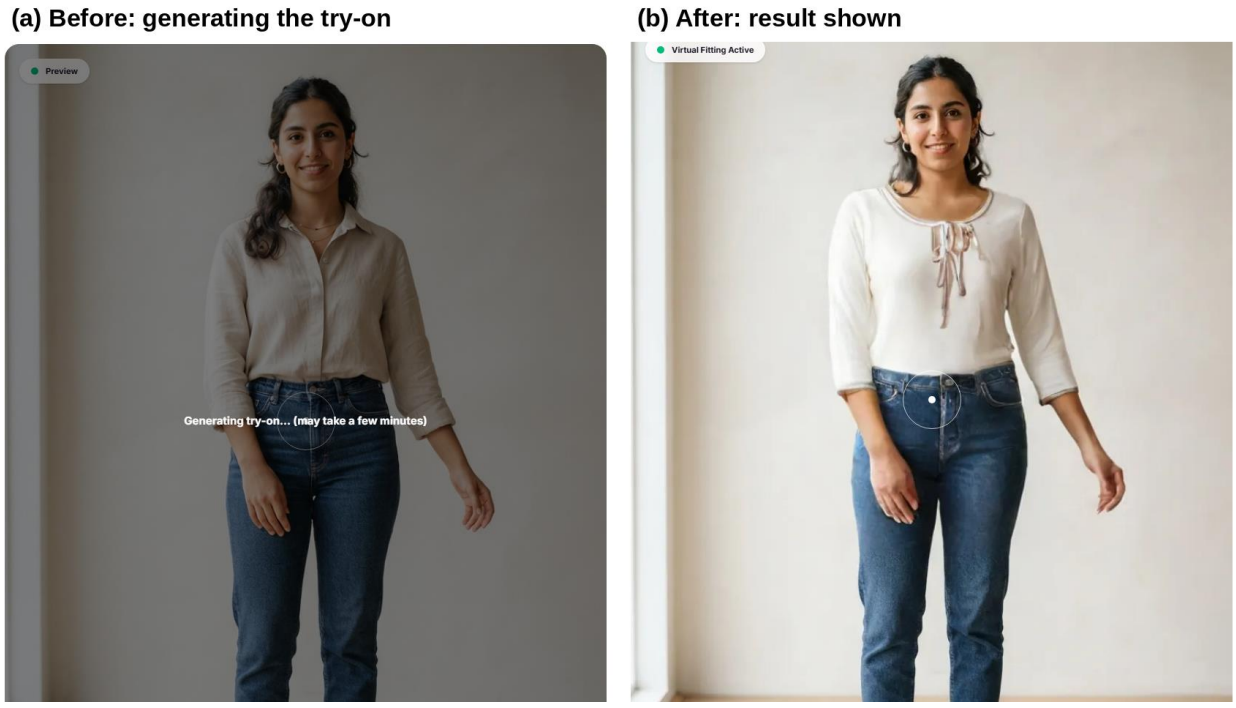


Figure 5.5. Try-on from the website: (a) the shopper's photograph while the try-on is being generated and (b) the result, in which the top of the selected outfit is rendered on the shopper.

Four limitations were observed. (i) The quality of the result depends on the resolution of the garment photograph, and the small images of the public catalogue give blurred garments. (ii) Layering two garments lowers the quality of the second step. (iii) A photograph that shows several garments needs the category to be given explicitly. (iv) Each generation takes minutes, and the public address of the service changes every time the notebook is started.

No image-quality metric was computed, because the shopper has no reference image, the model was not retrained and the number of runs is small. The results are therefore a demonstration that the module works inside the application on the shopper's own photograph, and not evidence of realism or of an effect on purchase decisions.

5.5 Summary and Threats to Validity

The recommendation experiments support three conclusions. First, rebalancing the compatibility network improved held-out accuracy from 0.646 to 0.789. Second, the per-item reuse cap increased outfit-set diversity with negligible relevance cost. Third, the end-to-end evaluation exposed a candidate-pool limitation that component-level tests did not, which the fallback ladder in the deployed system addresses. Four threats to validity remain. (i) Labels are catalogue-derived proxies, not human preferences. (ii) The labelled set is small (948 training pairs), so the neural network is not yet justified over the linear model. (iii) The catalogue mixes isolated product photographs with on-model photographs; embeddings of the latter encode the garment and the wearer together, and fashion CLIP models are reported to be biased towards centred product images on plain backgrounds [15]. (iv) Beam-search latency has not been benchmarked against the exhaustive pipeline. No human evaluation or public benchmark was used, so the results support the internal comparisons made here but not claims of superiority over the systems in Section 2.

For the body-measurement model the threats are as follows. The evidence is qualitative and comes from a few photographs without ground truth. The widths are measured from the front, so they are not girths [24], [25]. Errors in the stated height propagate to all widths, because the scale is derived from it [21].

For the virtual try-on model the threats are as follows. The evidence is qualitative and comes from a few runs with one shopper photograph, so different body shapes, skin tones, poses and garment types are not covered. The times come from single runs on a free GPU whose availability varies. The result depends on the resolution of the catalogue photographs, which in the public dataset are very small. And the website currently sends only the main garment, so the layered outfit of Section 3.4.4 was shown only in the notebook.

6. Discussion

6.1 Interpretation of the Compatibility Results

The most informative result is a negative one. After rebalancing, the dual-path network reached 0.789 test accuracy but remained well below the linear model (0.932), and the validation sweep gave the linear model full weight. Three observations explain this. First, the labelled set is small: 948 training pairs cannot support a network with a six-figure parameter count, which is the memorisation-versus-generalisation trade-off that motivated wide-and-deep models [9]. Second, the metadata-only model (0.928) is almost as accurate as the linear model with visual features (0.932), which suggests that the proxy labels depend mainly on category and colour. High accuracy therefore reflects, in part, the model recovering its own labelling rule rather than independent knowledge of what people consider well matched. Third, the visual signal alone is informative (0.705) but weaker, so a deep visual branch has little to add until more or better labels are available. For this reason the accuracy figures should be read as evidence that the pipeline learns what the labels encode, not as evidence about shopper taste.

6.2 Stability and Design Choices

The weight-sensitivity analysis shows that the identity of the top five outfits depends on the balance between relevance and compatibility. The compatibility-led default (0.6 and 0.4) and the 0.55 compatibility floor are therefore stated design decisions, not optima; validating them requires human preference data or implicit feedback such as clicks and cart additions. Following [12], the sensitivity is reported rather than hidden. Likewise, the reuse cap acts on product identifiers, so near-identical items under different identifiers can still repeat; a cap defined on embedding similarity is the natural next step.

6.3 Trade-offs of Pre-Computation

Pre-computing recommendations makes the application deterministic, fast and free to run, and it keeps price and stock live because they are joined at request time. The costs are three. The profile space is discrete: only body shape, size, occasion and season influence the outfits, so two shoppers with the same four values see the same five outfits, and finer measurements (for example a different waist-to-hip ratio within the same shape) do not change the result. Products added to the catalogue are not ranked until the index is regenerated. And the season defaults to the current date, so the same profile yields different outfits across the year, which is intended but should be communicated to the shopper. Serving the model live from a separate free-tier host may be possible, but it was not adopted or tested in this project, because it would add a second runtime to maintain.

6.4 Relation to Existing Work

The engine shares its main ingredients with recent systems: fashion CLIP embeddings and occasion priors expressed as prompts [8], and ranking that discourages repetition [7]. It differs in treating body shape, size availability, occasion and season as explicit inputs for a shop that can recommend only what is in stock in the shopper's size. It is weaker than the cited systems in its evaluation: no benchmark such as Polyvore and no human study was used, so superiority over them is not claimed.

6.5 Fairness, Inclusivity and Ethical Considerations

The body-shape rules (Appendix E) are common styling conventions chosen by the project team; they were not validated with stylists or shoppers and may encode normative assumptions about which silhouettes “suit” which bodies. They should be presented to shoppers as suggestions and be easy to override, which the interface supports by allowing any piece to be replaced. The catalogue stocks sizes S to XL only, so shoppers outside that range receive no exact-size recommendation, and mapping XS to S is an approximation. The catalogue and its labels come from a single public dataset, which may not represent the shop’s audience. Finally, recommendations depend on Model 1; photographs are personal data, and their handling is discussed in the section on Model 1.

6.6 Were the Objectives Met?

Table 6.1. Status of the project objectives.

Objective	Status	Evidence
O1 Platform	Implemented	Section 4.8
O2 Body measurement and size	Implemented; examined on a few test photographs, accuracy against tape measurements not yet measured	Sections 3.2 and 5.2
O3 Outfit recommendation	Met, with the limitations stated in Sections 5.5 and 6.1	Section 5.3
O4 Virtual try-on	Implemented as a queued GPU service called by the website; evidence is qualitative; takes minutes per outfit; renders tops, bottoms and dresses only	Sections 3.4, 4.5 and 5.4
O5 Integration	Implemented; error propagation between models not measured	Sections 3.1 and 4.5
O6 Low cost and reproducibility	Designed for free tiers; actual running cost not audited	Sections 3.6 and 4.6
O7 Evaluation	Partly met: no user study or public benchmark	Section 5.5

Source: this report (Sections 3 to 5).

6.7 Discussion of Models 1 and 3 and of the Integrated System

Model 1. The main strength of the module is that it is transparent: every step is a simple, inspectable computation, it needs no training data, it runs quickly and at no cost, and it reports how reliable its waist value is. The experiments show the value of diagnosing a baseline rather than adopting it: the landmark-based hip estimate was wrong because it measured a different quantity, and the segmentation-based waist search was wrong for loose clothing; each failure led to a specific correction. The same discipline produced the alignment of the body-shape classes with those of the recommendation engine, an interface defect that could not be seen from either component alone.

The limitations are equally clear. (i) The evidence is qualitative and drawn from a few photographs, without ground truth. (ii) The height must be supplied, and scale is derived from the nose-to-ankle distance, which biases all widths

upwards; monocular height inference is itself uncertain [21]. (iii) Widths seen from the front are not circumferences, so waist and hip sizes are underestimated for fuller bodies unless a conversion is applied, and arms close to the body inflate the hip width. (iv) The waist fallback recovers a position but not the true width. (v) The height offsets of the chest and thigh and the body-shape thresholds are empirical and have not been validated against experts. (vi) The size chart is a placeholder, only the shoulder width influences the size in the service, and the code contains more than one sizing rule. (vii) The photograph is personal data; it is processed by the measurement service and, if that service fails, it is sent to an external language-model service.

Comparison with the literature. Learning-based methods such as BMnet reach measurement accuracy that a hand-crafted pipeline cannot match, but they rely on a training set such as BodyM, on frontal and lateral silhouettes, and on height and weight as inputs [23]. The present pipeline occupies a different point in the design space: no training data, a single view, and complete transparency, with the cost that its accuracy is unknown until it is validated. Because FFIT classifications are sensitive to the definition of the measurements [24], the body-shape label of this module, which is derived from widths with uncalibrated thresholds, is treated as an approximate input: in the recommendation engine, body shape adds one of three equally weighted relevance terms and never excludes a product.

Model 3. The main strength of the module is that it gives the shopper a visual check of an outfit on her own photograph at no per-call cost, with an open model whose code and weights are public. The surrounding pipeline made it usable on shop photographs: garment cleaning removed mannequins and people from the product images, layering produced outfits of a top and a bottom, and the restoration step kept the face, hair and shoes of the shopper unchanged.

The limitations are as follows. (i) Each generation takes minutes, and the service depends on a free notebook GPU whose public address changes at every start and which disconnects after some time, so the set-up suits development and demonstration, not continuous production use. (ii) The quality depends on the photographs: small catalogue images give blurred garments, and a second layered garment lowers quality. (iii) Shoes and accessories are not rendered. (iv) The website sends only the main garment of the outfit. (v) The evidence is qualitative and comes from one shopper photograph. (vi) The shopper's photograph is personal data that is sent to an external GPU service; it is deleted when the job finishes, but the shopper should be told that it leaves the application. (vii) Dress Code was built from the catalogues of a single fashion retail group [26], so the fidelity for other body shapes, skin tones and garment styles is unknown and was not tested.

Comparison with the literature. IDM-VTON [19] and LaDI-VTON [20] report better results on benchmarks than earlier methods such as VITON [18], but both need a GPU. No comparison between them was run here, and LaDI-VTON was chosen for the openness of its code and weights and for its coverage of the three garment categories, not for a measured advantage.

A cross-cutting issue is error propagation. A wrong body-shape label or size from Model 1 changes every outfit shown by Model 2, and the garment image chosen by Model 2 limits what Model 3 can render. The quantitative effect of such errors across the three models has not been measured and is an open evaluation question.

7. Conclusion and Future Work

7.1 Conclusion

This work designed and built FitStyle AI, a low-cost web application that combines body measurement, outfit recommendation and virtual try-on in one shopping workflow. From a single full-body photograph and her height, the shopper receives a body shape and a clothing size, five complete outfits for her size, body shape, occasion and season, and a preview of the selected garments on her own photograph. Only the database, authentication and image storage are cloud services on free tiers, and the try-on model runs as a service on a free GPU notebook runtime. The evidence is uneven across the three components, and no user study was conducted, so claims about shopper preference remain open. The three AI components are summarised in turn.

Model 1. The body-measurement module shows that a transparent, training-free pipeline can turn one photograph and a stated height into shoulder, chest, waist, hip and thigh widths, a body shape and a size, and that experimental diagnosis of a landmark-based baseline is worthwhile: two weaknesses were found and corrected, and every waist estimate now carries a method and a confidence flag. Its accuracy has not yet been validated against tape measurements, and its size chart and body-shape thresholds are placeholders, so its output is best treated as a calibrated starting point that the shopper can edit.

Model 2. The recommendation engine designed and deployed for FitStyle AI composes complete outfits from a product catalogue for a given body shape, size, occasion and season. Diagnosing the compatibility network by ablation led to a rebalanced architecture whose test accuracy rose from 0.646 to 0.789; a beam-search construction with a per-item reuse cap nearly doubled the number of distinct items across the top five outfits at a negligible relevance cost; and an end-to-end evaluation exposed a sparse-candidate-pool limitation that was addressed by a fallback ladder. The validation-selected ensemble reduced to the linear model, a finding that reflects the scale and nature of the labels rather than a defect of the design. The engine is served to the web application at no additional cost by pre-computing its output and joining it with the live catalogue.

Model 3. The virtual try-on module shows that an open latent-diffusion model, LaDI-VTON, can be placed behind a shop workflow at no per-call cost by adding garment cleaning, person preparation, sequential layering and identity restoration around it and by serving it from a free GPU as a queued service. In the notebook runs and in the website it rendered tops, bottoms and a dress on the shopper's photograph and kept her face, hair and shoes. Each generation takes minutes, the evidence is qualitative and comes from one shopper photograph, and the website currently sends only the main garment of the outfit.

7.2 Future Work

Future work is organised by component, in the order of the shopper's journey: the body-measurement module (Model 1), the recommendation engine (Model 2) and the virtual try-on module (Model 3), followed by the platform as a whole.

7.2.1 Model 1: Body-Measurement Module

The following steps are proposed. (i) Validate the module against tape measurements taken according to ISO 8559-1 [25] on a larger and more varied group, and calibrate the height offsets, the body-shape thresholds and the size chart on that data. (ii) Estimate girths instead of widths, for example by adding a lateral photograph, as

BodyM does [23]. (iii) Use the world-coordinate landmarks that the Pose Landmarker already returns [22] to obtain a metric reference that does not depend on the nose-to-ankle distance. (iv) Detect the garment type in advance so that unreliable cases are flagged before measurement. (v) Replace the placeholder size chart with brand charts and let hip width influence the size.

7.2.2 Model 2: Outfit Recommendation Engine

For the recommendation engine the following steps are proposed.

- Collect human preference or implicit-feedback data (ratings, clicks, cart additions) to replace the proxy labels and to validate the weighting, the compatibility floor and the body-shape rules.
- Revisit the neural branch once the labelled set is larger, or use self-supervised pre-training to reduce the dependence on labels.
- Define the reuse cap on embedding similarity rather than product identity, and widen the candidate pool adaptively for sparse profiles.
- Benchmark beam-search latency against exhaustive search and evaluate on a public outfit-compatibility benchmark.
- Replace pre-computation by a live service if new products must be ranked immediately, and make the profile continuous by using measurements beyond shape and size.
- Extend the size range beyond S to XL and replace the XS-to-S mapping with real stock.

7.2.3 Model 3: Virtual Try-On Module

The following steps are proposed. (i) Evaluate the module with a reference-free image-quality metric and with a user study that includes several shoppers, body shapes and skin tones. (ii) Send the complete top-and-bottom outfit from the website and measure how much quality the second step loses. (iii) Host the service on a persistent GPU with a stable address instead of a temporary notebook address, and cache the results of outfits that are requested repeatedly. (iv) Reduce the latency, for example with fewer denoising steps or a smaller image, and compare LaDI-VTON with IDM-VTON [19] on the same inputs. (v) Require garment photographs of sufficient resolution and use the garment category stored in the catalogue instead of detecting it.

7.2.4 Platform as a Whole

For the platform as a whole the following steps are proposed.

- Harden the remaining server endpoints (product management, user profile, measurements, order history, analytics) with authentication, ownership and administrator checks.
- Measure the effect of error propagation across the three models, for example how a wrong body-shape label or size from Model 1 changes the outfits shown by Model 2, and how the garment image chosen by Model 2 limits what Model 3 can render.

References

[1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in Proc. Int. Conf. Machine Learning (ICML), 2021, pp. 8748–8763. arXiv:2103.00020. Available: https://arxiv.org/abs/2103.00020

[2] P. J. Chia, G. Attanasio, F. Bianchi, S. Terragni, A. R. Magalhães, D. Goncalves, C. Greco, and J. Tagliabue, "Contrastive language and vision learning of general fashion concepts," Scientific Reports, vol. 12, art. 18958, 2022, doi: 10.1038/s41598-022-23052-9. Available: https://doi.org/10.1038/s41598-022-23052-9

[3] Marqo, "Marqo-FashionCLIP model card," Hugging Face. Fine-tuned from ViT-B-16 (laion2b_s34b_b88k) with Generalised Contrastive Learning. Accessed: Oct. 2026. Available: https://huggingface.co/Marqo/marqo-fashionCLIP

[4] X. Han, Z. Wu, Y.-G. Jiang, and L. S. Davis, "Learning Fashion Compatibility with Bidirectional LSTMs," in Proc. ACM Int. Conf. Multimedia (MM), 2017. arXiv:1707.05691. Available: https://arxiv.org/abs/1707.05691

[5] M. I. Vasileva et al., "Learning Type-Aware Embeddings for Fashion Compatibility," in Proc. European Conf. Computer Vision (ECCV), 2018. arXiv:1803.09196. Available: https://arxiv.org/abs/1803.09196

[6] K. Kalashi and B. Teimourpour, "A Hybrid Multimodal Deep Learning Framework for Intelligent Fashion Recommendation," arXiv:2511.07573, 2025. Available: https://arxiv.org/abs/2511.07573

[7] T. Koukopoulos, D. Klimenof, and I. Xarchakos, "TrendGen: An Outfit Recommendation and Display System," arXiv:2603.27264, Mar. 2026. Available: https://arxiv.org/abs/2603.27264

[8] A. Berlia, "Loom: Hybrid Retrieval-Scoring Outfit Recommendation with Semantic Material Compatibility and Occasion-Aware Embedding Priors," arXiv:2605.09830, May 2026. Available: https://arxiv.org/abs/2605.09830

[9] H.-T. Cheng et al., "Wide & Deep Learning for Recommender Systems," in Proc. 1st Workshop on Deep Learning for Recommender Systems (DLRS), 2016, pp. 7–10, doi: 10.1145/2988450.2988454. Available: https://doi.org/10.1145/2988450.2988454

[10] C. Meng, J. Luo, P. Feng, Z. Yan, C. Hu, Z. Gan, and C. Ouyang, "Mitigating Shared-Private Branch Imbalance via Dual-Branch Rebalancing for Multimodal Sentiment Analysis," arXiv:2604.25179, 2026 (accepted at ACM Multimedia 2026). Available: https://arxiv.org/abs/2604.25179

[11] A. K. Vijayakumar, M. Cogswell, R. R. Selvaraju, Q. Sun, S. Lee, D. Crandall, and D. Batra, "Diverse Beam Search for Improved Description of Complex Scenes," in Proc. AAAI Conf. Artificial Intelligence, vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.12340. Available: https://doi.org/10.1609/aaai.v32i1.12340

[12] R. O'Shea, P. Deeney, E. Triantaphyllou, L. Diaz-Balteiro, and S. A. Tarim, "Weight stability intervals for multi-criteria decision analysis using the weighted sum model," Expert Systems with Applications, vol. 296, art. 128460, 2026, doi: 10.1016/j.eswa.2025.128460. Available: https://doi.org/10.1016/j.eswa.2025.128460

[13] A.-S. Sheikh, R. Guigourès, E. Koriagin, Y. K. Ho, R. Shirvany, R. Vollgraf, and U. Bergmann, "A Deep Learning System for Predicting Size and Fit in Fashion E-Commerce," in Proc. ACM Conf. Recommender Systems (RecSys), 2019. arXiv:1907.09844. Available: https://arxiv.org/abs/1907.09844

[14] ashraq, "fashion-product-images-small," Hugging Face dataset (user ashraq). Accessed: Oct. 2026. Available: https://huggingface.co/datasets/ashraq/fashion-product-images-small

[15] P. J. Chia et al., "fashion-clip model card," Hugging Face (describes FashionCLIP training data and noted limitations). Accessed: Oct. 2026. Available: https://huggingface.co/patrickjohncyh/fashion-clip

[16] V. Bazarevsky et al., "BlazePose: On-device Real-time Body Pose tracking," arXiv:2006.10204, 2020. Available: https://arxiv.org/abs/2006.10204

[17] S. Bai et al., "Qwen2.5-VL Technical Report," arXiv:2502.13923, 2025. Available: https://arxiv.org/abs/2502.13923

[18] X. Han et al., "VITON: An Image-Based Virtual Try-On Network," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7543–7552. arXiv:1711.08447. Available: https://arxiv.org/abs/1711.08447

[19] Y. Choi et al., “Improving Diffusion Models for Authentic Virtual Try-on in the Wild,” in Proc. European Conf. Computer Vision (ECCV), 2024. arXiv:2403.05139. Available: https://arxiv.org/abs/2403.05139

[20] D. Morelli, A. Baldrati, G. Cartella, M. Cornia, M. Bertini, and R. Cucchiara, “LaDI-VTON: Latent Diffusion Textual-Inversion Enhanced Virtual Try-On,” in Proc. 31st ACM Int. Conf. Multimedia (MM ’23), 2023, pp. 8580–8589, doi: 10.1145/3581783.3612137. arXiv:2305.13501. Available: https://doi.org/10.1145/3581783.3612137

[21] S. Günel, H. Rhodin, and P. Fua, “What Face and Body Shapes Can Tell Us About Height,” in Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW), 2019, doi: 10.1109/ICCVW.2019.00226. arXiv:1805.10355. Available: https://doi.org/10.1109/ICCVW.2019.00226

[22] Google AI Edge, “Pose landmark detection guide,” MediaPipe Solutions documentation (Pose Landmarker task: 33 landmarks, world coordinates and optional segmentation mask). Accessed: Oct. 2026. Available: https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

[23] N. Ruiz et al., “Human Body Measurement Estimation with Adversarial Augmentation,” in Proc. Int. Conf. 3D Vision (3DV), 2022. arXiv:2210.05667. Available: https://arxiv.org/abs/2210.05667

[24] C. J. Parker, S. G. Hayes, K. Brownbridge, and S. Gill, “Assessing the female figure identification technique’s reliability as a body shape classification system,” Ergonomics, vol. 64, no. 8, pp. 1035–1051, 2021, doi: 10.1080/00140139.2021.1902572. Available: https://doi.org/10.1080/00140139.2021.1902572

[25] ISO, “ISO 8559-1:2017, Size designation of clothes — Part 1: Anthropometric definitions for body measurement,” International Organization for Standardization, 2017 (confirmed current in 2026). Available: https://www.iso.org/standard/61686.html

[26] D. Morelli, M. Fincato, M. Cornia, F. Landi, F. Cesari, and R. Cucchiara, “Dress Code: High-Resolution Multi-Category Virtual Try-On,” in Proc. European Conf. Computer Vision (ECCV), 2022, doi: 10.1007/978-3-031-20074-8_20. Available: https://doi.org/10.1007/978-3-031-20074-8_20

[27] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022, pp. 10684–10695. arXiv:2112.10752. Available: https://arxiv.org/abs/2112.10752

[28] miccunifi, “ladi-vton: official repository of LaDI-VTON,” GitHub. Accessed: Oct. 2026. Available: https://github.com/miccunifi/ladi-vton

Appendices

Appendix A. System Architecture and Recommendation Interfaces

The recommendation engine is exposed to the web application through three authenticated endpoints. Recommendations are looked up from the pre-computed index and joined with the live catalogue; no neural inference occurs at request time.

Table A.1. Recommendation interfaces.

Endpoint	Input	Output
GET /api/recommendations	size, body shape, occasion, season (optional; defaults to the current season)	Up to five outfits; each with its pieces (live price, colour, size, image), match score and explanation
GET /api/recommendations/alternatives	Profile and a garment category	Other suitable products of that category in the shopper's size, ranked by relevance
POST /api/analyze-body	Photograph and height	Widths, body shape, size and annotated photograph (Appendix J)
POST /api/recommendations/explain	Profile and the identifiers of the currently selected pieces	Headline, explanation and a one-line rationale for exactly that selection

Source: server code of the application.

Shopper profile → look-up of pre-computed outfits → join with DynamoDB-backed catalogue (stock, size, price) → presentation of five outfits → optional replacement of a single piece from the category pool → regenerated justification → add to cart. The cart is stored per user and is emptied after a successful payment.

Appendix B. Recommendation Pipeline Configuration

Table B.1. Parameters of the recommendation pipeline.

Parameter	Value	Meaning
Embedding model	Marqo-FashionCLIP (frozen)	Image and text embeddings in one space [3]
Image / text fusion weights	0.7 / 0.3	Weights used to combine item image and text embeddings
Relevance weights (style, body shape, occasion)	1/3 each	Equal weighting of the three relevance signals
Occasion penalty factor λ	2.0	Penalty on similarity to the anti-vibe prompt
Candidates per core category / optional slot	12 / 8	Pool sizes before outfit assembly
Beam width	30	Partial outfits kept per search step
Outfit score weights	0.6 compatibility, 0.4 relevance	Compatibility-led default
Colour penalty weight	0.1	Penalty for non-neutral colour clashes

Parameter	Value	Meaning
Minimum compatibility floor	0.55	Outfits whose weakest pairwise link is lower are dropped
Maximum item reuse	2	Maximum number of top outfits that may share one item
Train / validation / test fractions	0.70 / 0.15 / 0.15	Product-level split
Optimiser	Adam, lr 1×10^{-3}, batch 64, weight decay 3×10^{-4}	Compatibility network training
Epochs / patience / dropout	up to 120 / 12 / 0.2	Early stopping on validation loss
Random seed	42	Python, NumPy and PyTorch

Source: configuration block of the recommendation notebook.

Appendix C. Pipeline Summary

For a profile (size, body shape, occasion, season, style) the engine proceeds as follows. (1) Filter the catalogue to products available in the shopper's size and matching the season and occasion (with the fallback ladder of Section 5.3.6). (2) Embed each remaining product and compute three relevance signals: similarity to a style–season prompt; a body-shape score equal to the maximum similarity to "preferred" silhouette prompts minus the maximum similarity to "avoided" prompts; and an occasion affinity equal to similarity to the occasion prompt minus a penalty for similarity to the contrasting prompt. (3) Average the three signals into a relevance score and keep the best candidates per garment category. (4) Assemble outfits from category templates with beam search, scoring each partial outfit with 0.6 × mean pairwise compatibility + 0.4 × mean relevance − 0.1 × colour penalty. (5) Discard outfits whose weakest pairwise compatibility is below 0.55. (6) Select the final five outfits in score order, skipping any outfit that exceeds the similarity threshold with an earlier outfit or would reuse an item more than twice. (7) Generate an explanation from structured evidence.

Appendix D. Catalogue Description

Table D.1. Catalogue composition by garment category.

Garment category	Products
Accessory	4,683
Top	2,939
Shoes	2,799
Bottom	497
Dress	390
Outerwear	205
Total	11,513

Source: catalogue loaded in the recommendation notebook from the dataset [14].

Attributes used are the product identifier, gender, article type, base colour, usage, season and display name. The source dataset is [14]. Available sizes and prices were generated for the DynamoDB catalogue by a seeded script, because the source dataset provides neither; the recommendation export reads the sizes actually stored in DynamoDB so that no recommendation contains a size the shop does not hold.

Appendix E. Body-Shape and Occasion Prompts

Table E.1. Silhouette preferences used for the body-shape score.

Body shape	Preferred silhouettes	Avoided silhouettes
Pear	A-line skirt or dress; wrap top or dress; flared silhouette; boat-neck or off-shoulder top	Bodycon fit; pencil skirt; skinny tapered bottoms
Hourglass	Fitted, belted silhouette; wrap dress; high-waisted cut; V-neck top	Oversized boxy fit; shapeless silhouette
Apple	Empire waist; A-line silhouette; flowy tunic; V-neck top	Bodycon fit; cropped tight top
Rectangle	Peplum top; belted waist; ruffled or layered design; flared silhouette	Straight, shapeless cut
Inverted triangle	Flared or wide-leg bottom; A-line silhouette; bootcut fit	Boat-neck top; puff sleeves; shoulder pads

Source: design rules chosen by the project team; they were not validated with stylists.

The silhouette preferences are design rules chosen by the project team from common styling guidance; they were not validated against expert stylists or user outcomes. Each occasion is represented by a descriptive “vibe” prompt and a contrasting “anti-vibe” prompt (for example, casual: “relaxed everyday casual comfortable easygoing weekend cotton denim sneakers” versus “formal black tie gown tuxedo evening wear”). The full prompt set is in the project notebook.

Appendix F. Reproducibility

- Catalogue: Hugging Face dataset ashraq/fashion-product-images-small [14].
- Embedding model: Marqo/marqo-fashionCLIP loaded through OpenCLIP, used frozen [3].
- Seeds: 42 for Python, NumPy and PyTorch. The export of Section 5.3.6 is deterministic given the same catalogue and sizes.
- Try-on model: LaDI-VTON, official repository miccunifi/ladi-vton [28], with the Dress Code weights; Stable Diffusion 2 inpainting weights from a Hugging Face mirror; the notebook FitStyle_TryOn_LaDI_VTON, run on a Colab T4 GPU (Appendices G and K).
- Artefacts: the project notebook (recommendation engine v20 and the website-export extension), the exported recommendation index, and the server module that serves it.

Appendix G. Appendices for Models 1 and 3

Table G.1. Parameters of the body-measurement model (from the notebook and the service file).

Parameter	Value
Pose model	MediaPipe Pose Landmarker, heavy variant (float16), with segmentation mask
Landmarks used	0 (nose), 11 and 12 (shoulders), 23 and 24 (hips), 25 (left knee), 27 (left ankle)
Scale	stated height / pixel distance between nose and left ankle; default height 165 cm
Mask threshold	0.5
Chest height	mean shoulder height + 0.05 of the image height
Thigh height	mean height of landmarks 23 and 25 + 0.03 of the image height
Waist search window	from 0.15 T below the shoulders to 0.10 T above the hips (T = torso height)
Waist acceptance range	relative position 0.35 to 0.65 of the torso; otherwise 0.55 and confidence low
Body-shape rules (centimetres)	D = S − H, M = min(S, H) − W. Rectangle if |D| < 10 and |W − H| < 10; hourglass if |D| < 8 and M ≥ 15; apple if |W − H| < 10 and M < 15; pear if H − S ≥ 8; inverted triangle if S − H ≥ 8; otherwise rectangle
Shoulder-width size chart (cm)	S up to 40, M up to 44, L up to 48, otherwise XL (placeholder; hip limits of 90, 100 and 110 are defined but not used)
Image padding	right edge replicated until the width is a multiple of four pixels
Service endpoint and port	POST detect-size and GET health on port 5001; fallback after 15 seconds

Source: Model 1 notebook and service file.

Table G.2. Parameters of the virtual try-on model and service (from the notebook).

Parameter	Value
Model	LaDI-VTON, official repository miccunifi/ladi-vton, with the pre-trained Dress Code weights for upper_body, lower_body and dresses
Diffusion backbone	Stable Diffusion 2 inpainting, loaded from the mirror alwold/stable-diffusion-2-inpainting because the original repository was removed
Python environment	Python 3.10 virtual environment; torch 2.0.1, torchvision 0.15.2, diffusers 0.14.0, transformers 4.27.3, accelerate 0.18.0, huggingface_hub 0.14.1, opencv-python 4.7.0.72, mediapipe 0.10.14
Hardware	Google Colab with a T4 GPU
Inference settings	dataset dresscode; test order unpaired; mixed precision fp16; batch size 1; 2 workers; PNG output
Input format	Person and garment images of 768 × 1024 pixels; garment mask; body-part label map; 18 COCO keypoints in a 384 × 512 space
Garment parsing	segformer_b2_clothes (mattmdjaga/segformer_b2_clothes); rembg mask as a fallback; images with a short side below 512 pixels are enlarged and those below 300 pixels are flagged as low resolution

Parameter	Value
Pose keypoints	MediaPipe pose landmarks mapped to the COCO-18 layout; landmarks with visibility below 0.3 are discarded; the neck is the midpoint of the shoulders
Label merging	lower_body: skirt merged into pants; dresses: upper clothes, skirt, pants and belt merged into dress
Outfit order	lower_body first, then upper_body; a dress is processed alone and the other garments are ignored
Identity restoration	Labels 1, 2, 3, 9, 10 and 11 (hat, hair, sunglasses, shoes and face) are copied back from the original photograph through a mask blurred with a Gaussian of radius 2

Source: Model 3 notebook.

Appendix H. Application Screens

This appendix reproduces the screens of the application that are not shown in Chapter 4. Together with Figures 4.2 to 4.6 and 5.5 they cover the complete journey of a shopper and the two dashboards of the store owner (Table 4.3), and they serve as a short user guide.

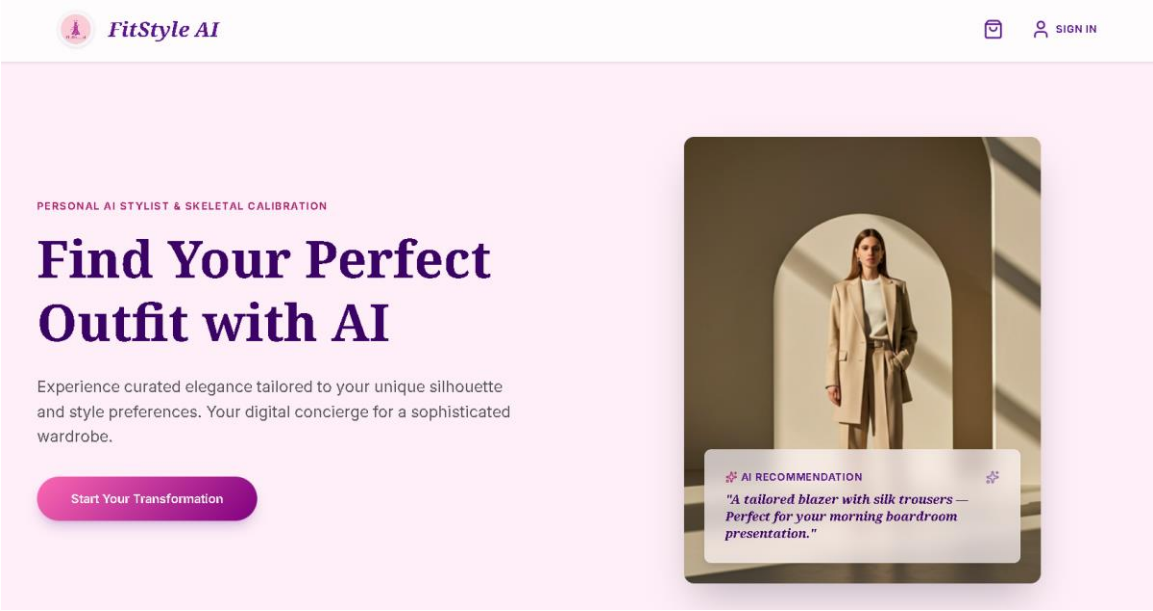


Figure H.1. Landing page.

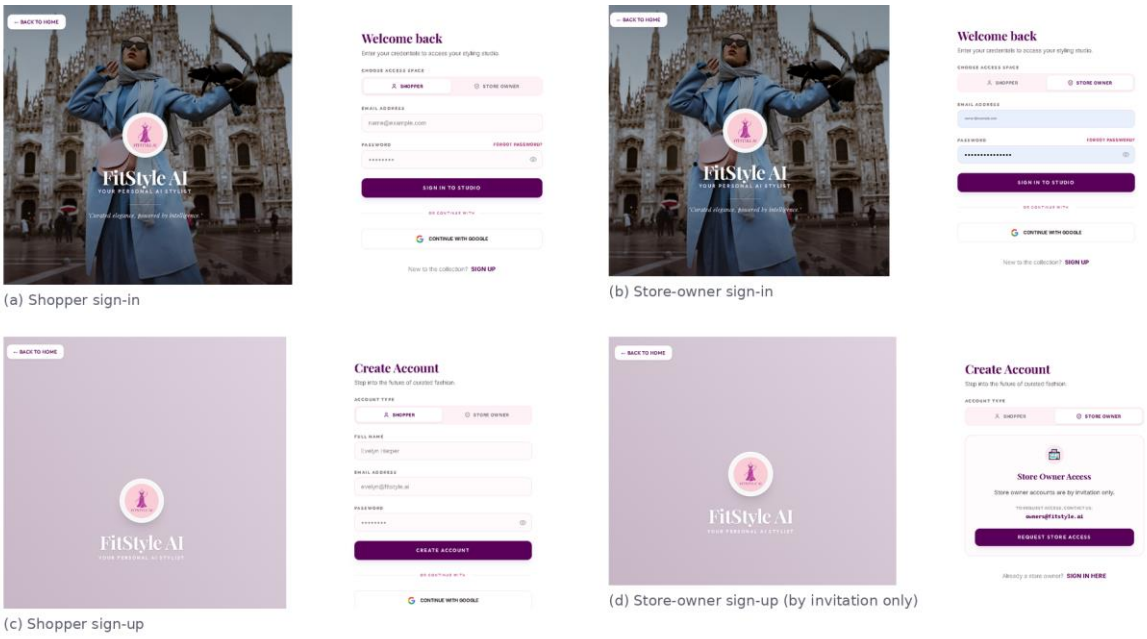


Figure H.2. Authentication screens: (a) shopper sign-in, (b) store-owner sign-in, (c) shopper sign-up and (d) store-owner sign-up, which is by invitation only.

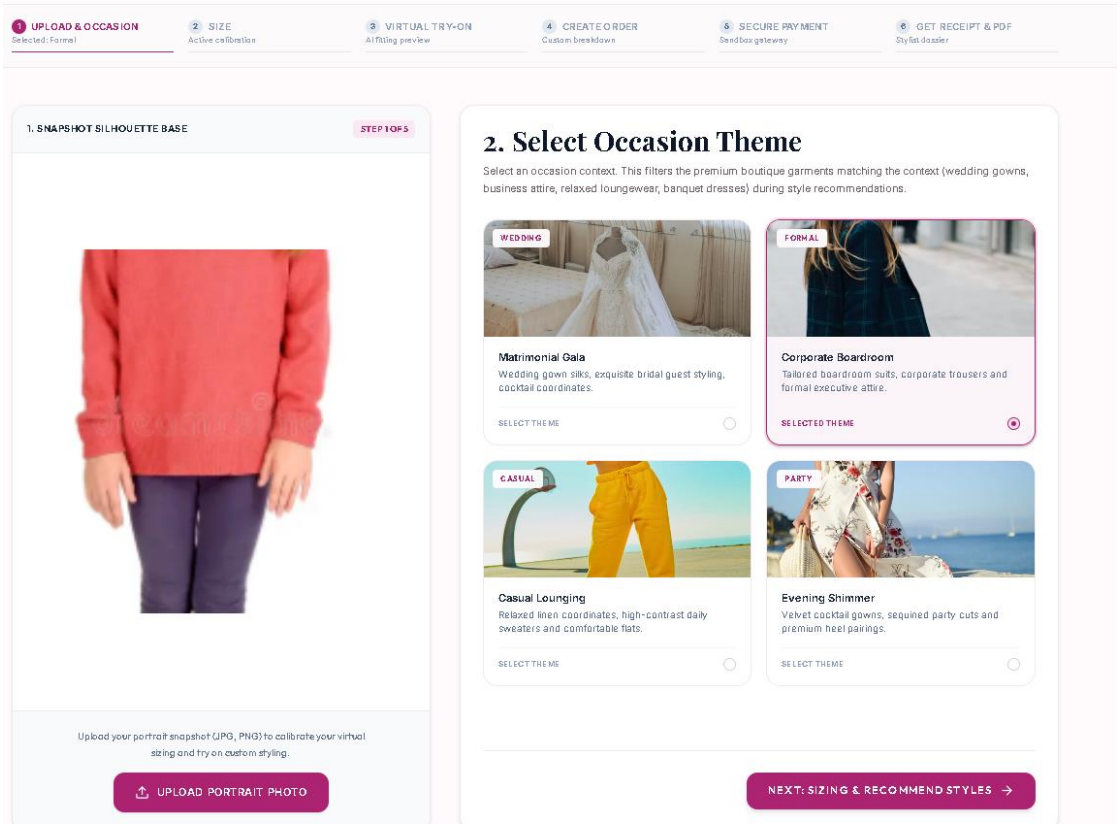


Figure H.3. Step 1: upload of the photograph and choice of the occasion.

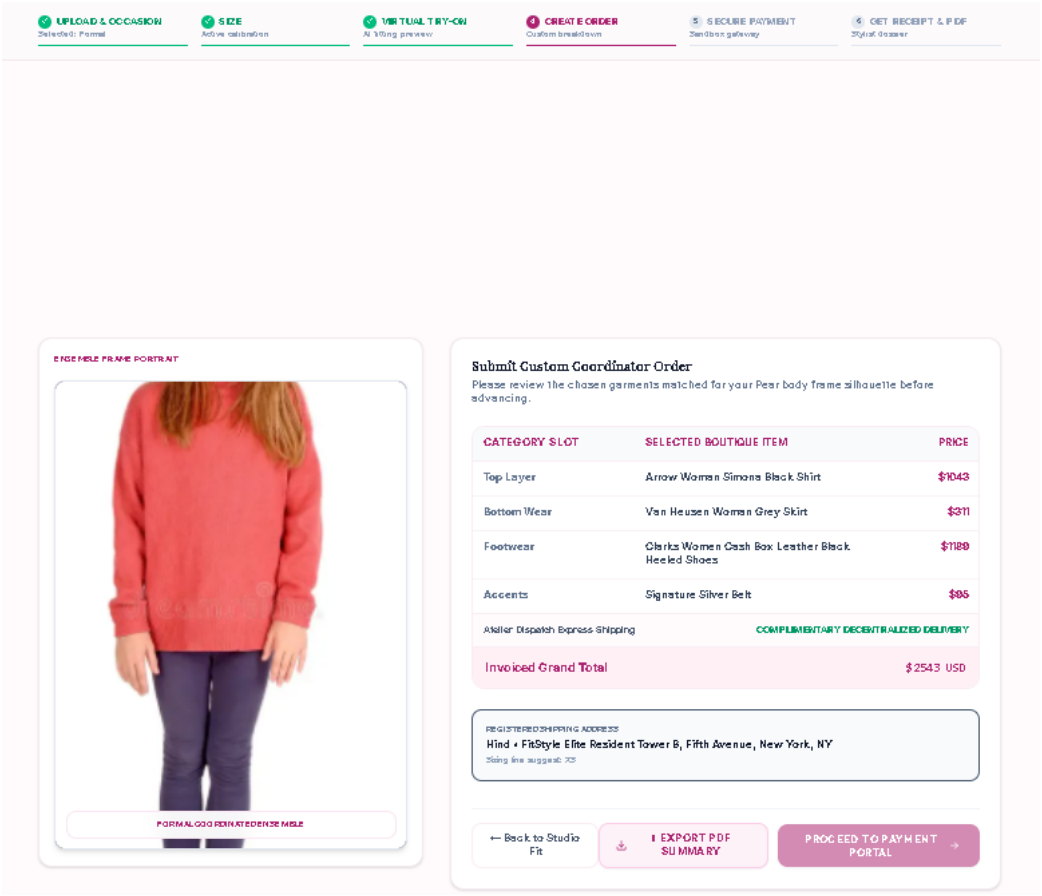


Figure H.4. Step 4: creation of the order, listing the chosen pieces by category slot, with export of a PDF summary.

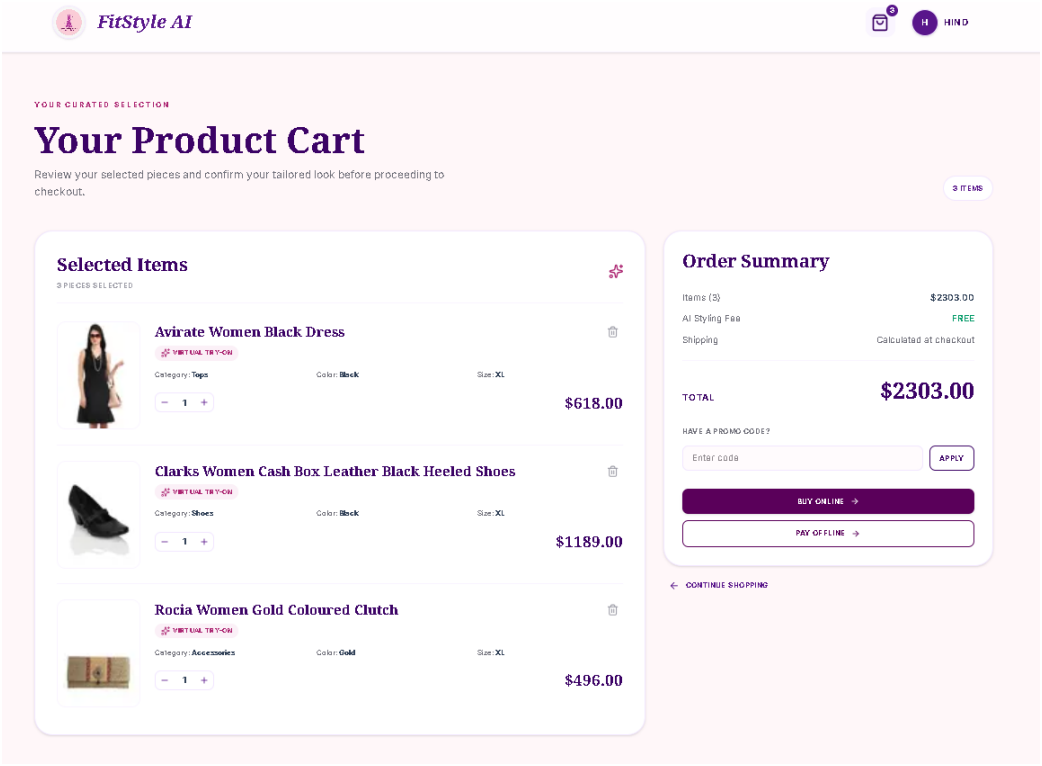


Figure H.5. Shopping cart with the selected pieces, the order summary, a promo-code field and the choice between buying online and paying offline.

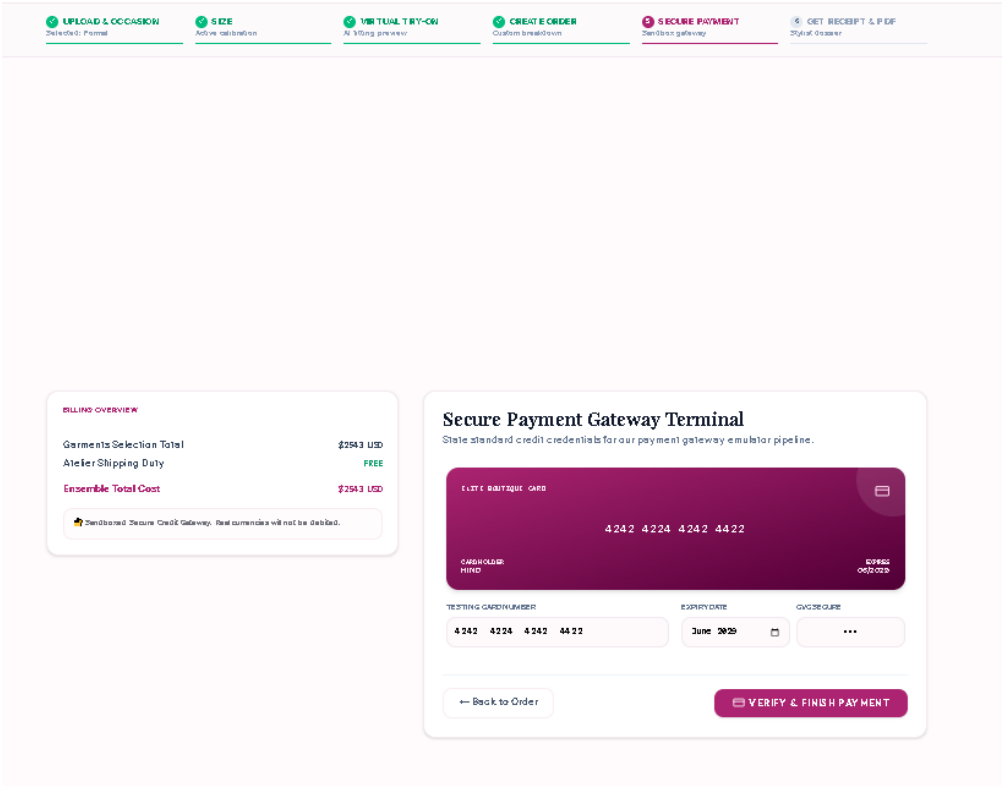


Figure H.6. Step 5: payment in the sandbox gateway with a test card.

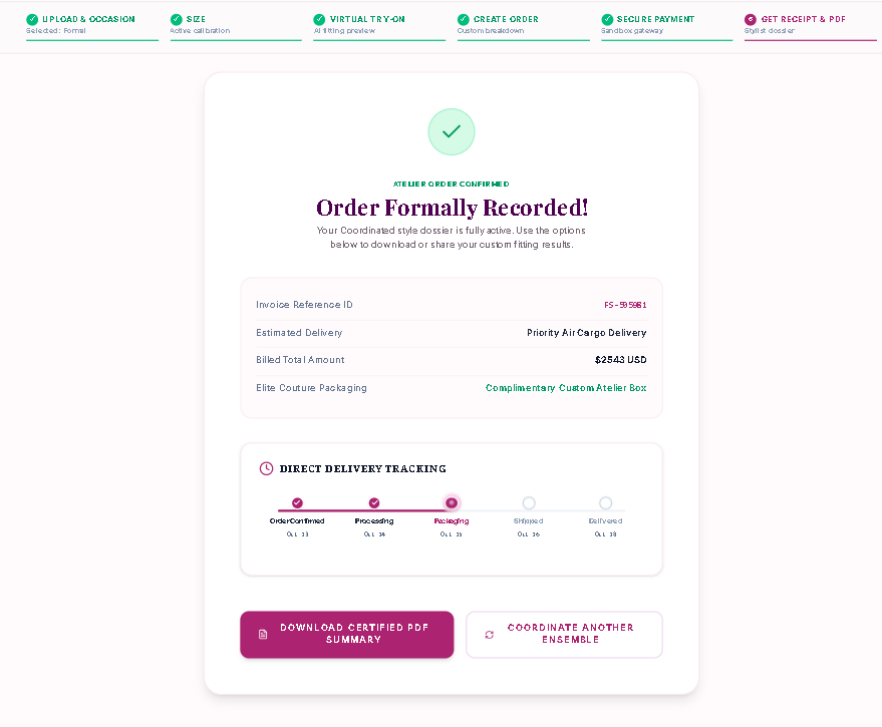
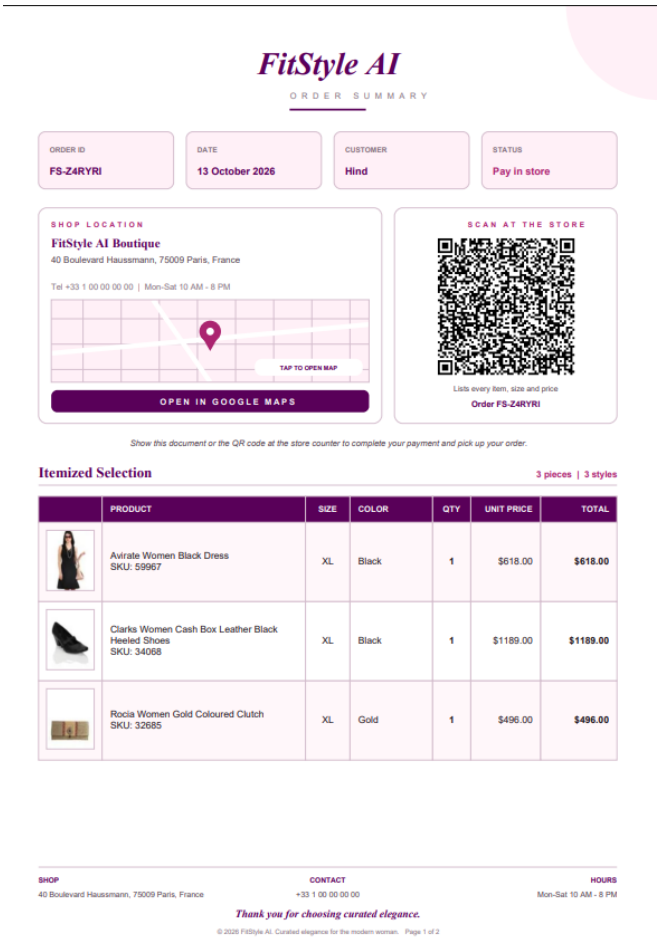


Figure H.7. Step 6: order confirmation with delivery tracking and the download of the PDF summary.



FitStyle AI

ORDER SUMMARY

ORDER ID	DATE	CUSTOMER	STATUS
FS-Z4RYRI	13 October 2026	Hind	Pay in store

SHOP LOCATION

FitStyle AI Boutique

40 Boulevard Haussmann, 75009 Paris, France

Tel +33 1 00 00 00 00 | Mon-Sat 10 AM - 8 PM

TAP TO OPEN MAP

OPEN IN GOOGLE MAPS

SCAN AT THE STORE

Lists every item, size and price

Order FS-Z4RYRI

Show this document or the QR code at the store counter to complete your payment and pick up your order.

Itemized Selection — 3 pieces | 3 styles

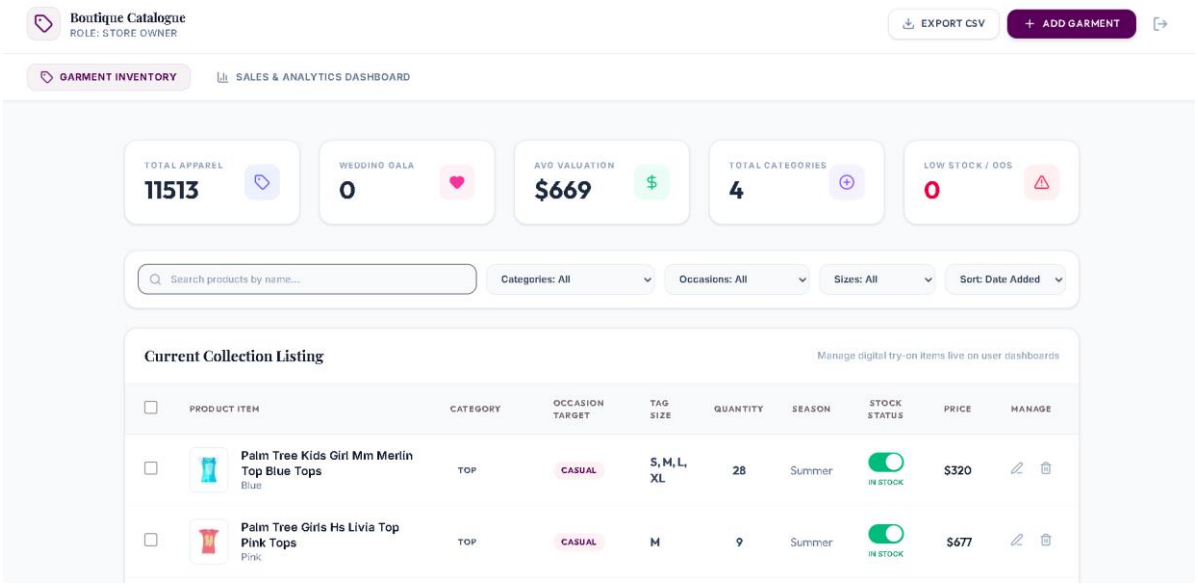
	PRODUCT	SIZE	COLOR	QTY	UNIT PRICE	TOTAL
	Avirate Women Black Dress SKU: 59967	XL	Black	1	$618.00	$618.00
	Clarks Women Cash Box Leather Black Heeled Shoes SKU: 34068	XL	Black	1	$1189.00	$1189.00
	Rocia Women Gold Coloured Clutch SKU: 32685	XL	Gold	1	$496.00	$496.00

SHOP	CONTACT	HOURS
40 Boulevard Haussmann, 75009 Paris, France	+33 1 00 00 00 00	Mon-Sat 10 AM - 8 PM

Thank you for choosing curated elegance.

© 2026 FitStyle AI. Curated elegance for the modern woman. Page 1 of 2

Figure H.8. PDF summary of an order paid offline, with the QR code and the shop location.



Boutique Catalogue

ROLE: STORE OWNER

EXPORT CSV | + ADD GARMENT

GARMENT INVENTORY | SALES & ANALYTICS DASHBOARD

TOTAL APPAREL	WEDDING GALA	AVG VALUATION	TOTAL CATEGORIES	LOW STOCK / OOS
11513	0	$669	4	0

Search products by name... | Categories: All | Occasions: All | Sizes: All | Sort: Date Added

Current Collection Listing

Manage digital try-on items live on user dashboards

PRODUCT ITEM	CATEGORY	OCCASION TARGET	TAG SIZE	QUANTITY	SEASON	STOCK STATUS	PRICE	MANAGE
Palm Tree Kids Girl Mm Merlin Top Blue Tops — Blue	TOP	CASUAL	S, M, L, XL	28	Summer	IN STOCK	$320	
Palm Tree Girls Hs Livia Top Pink Tops — Pink	TOP	CASUAL	M	9	Summer	IN STOCK	$677	

Figure H.9. Store owner's inventory dashboard.

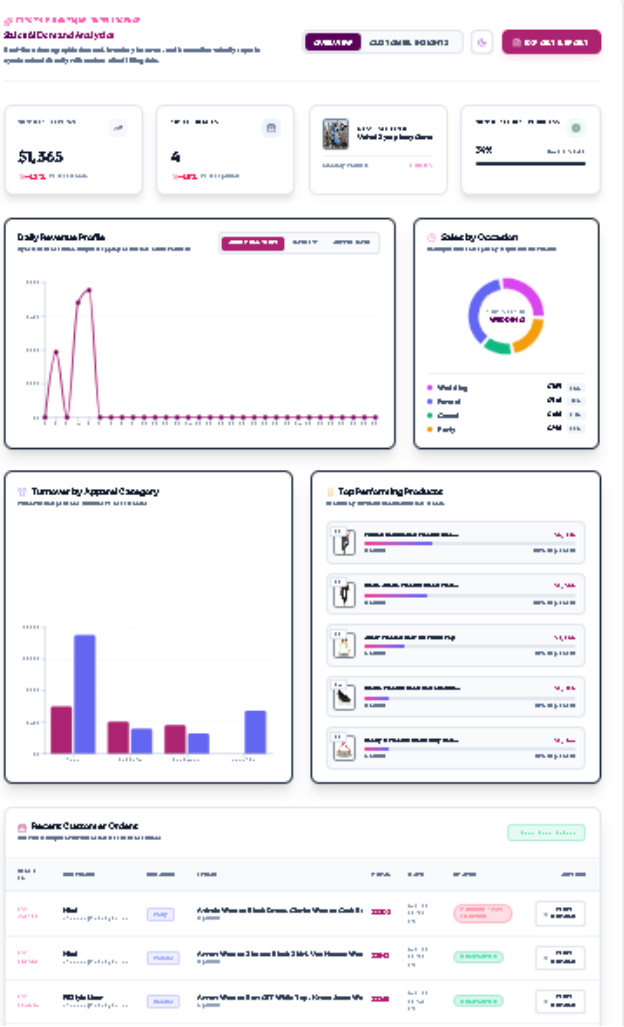


Figure H.10. Store owner's sales and analytics dashboard.

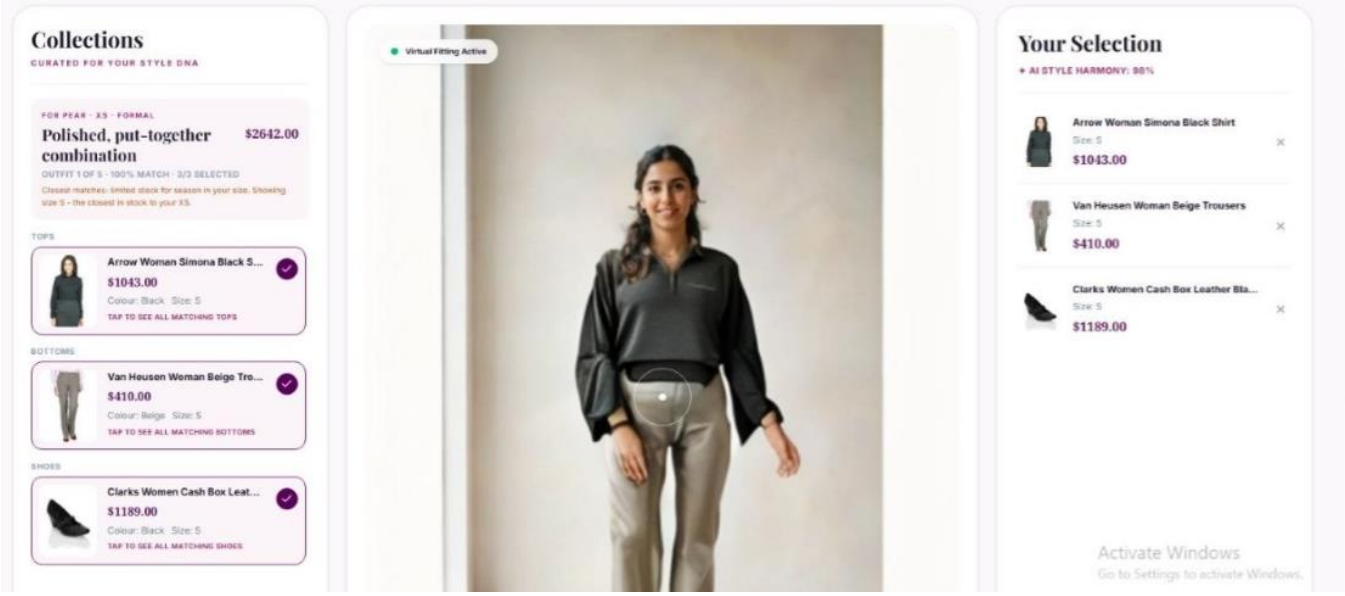


Figure H.11. Try-on step after generation: the preview shows the shopper wearing the black top and beige trousers of the selected outfit, and the status label reads "Virtual Fitting Active".

Appendix I. MediaPipe Pose Landmarks

The Pose Landmarker returns 33 landmarks per person in normalised image coordinates and in world coordinates [22]. Table I.1 lists the index and the body part of each landmark and marks those used by the body-measurement model.

Table I.1. The 33 pose landmarks and their use in the body-measurement model.

Index	Landmark	Used	Index	Landmark	Used
0	Nose	scale	17	Left pinky	no
1	Left eye (inner)	no	18	Right pinky	no
2	Left eye	no	19	Left index	no
3	Left eye (outer)	no	20	Right index	no
4	Right eye (inner)	no	21	Left thumb	no
5	Right eye	no	22	Right thumb	no
6	Right eye (outer)	no	23	Left hip	hip, waist, thigh
7	Left ear	no	24	Right hip	hip, waist
8	Right ear	no	25	Left knee	thigh
9	Mouth (left)	no	26	Right knee	no
10	Mouth (right)	no	27	Left ankle	scale
11	Left shoulder	shoulder, chest, waist	28	Right ankle	no
12	Right shoulder	shoulder, chest, waist	29	Left heel	no
13	Left elbow	no	30	Right heel	no

Index	Landmark	Used	Index	Landmark	Used
14	Right elbow	no	31	Left foot index	no
15	Left wrist	no	32	Right foot index	no
16	Right wrist	no			

Source: MediaPipe Pose Landmarker documentation [22]; the last column describes the use in this project.

Appendix J. Body-Measurement Service Interface

Table J.1. Request and response of the body-measurement service (from the service file).

Item	Field	Content
Request (POST, JSON)	imageBase64	Photograph as a base64 string; a data-URL prefix is accepted
Request (POST, JSON)	heightCm	Height in centimetres; 165 if omitted
Response	measurements	shoulder, chest, waist, hip and thigh widths and height_cm, in centimetres (a width is null if it cannot be read)
Response	body_shape	pear, hourglass, apple, rectangle, inverted_triangle or unknown
Response	recommended_size	S, M, L or XL (shoulder-width chart)
Response	waist_method and waist_confidence	segmentation or proportional_fallback; high or low
Response	line_positions	Normalised vertical positions (0 to 1) of the shoulder, waist and hip lines
Response	annotated_image_base64	JPEG of the photograph with the skeleton and the three labelled reference lines
Errors	400, 422, 500	Missing image; no person detected or undecodable image; any other failure

Source: service file of the body-measurement service.

Appendix K. Virtual Try-On Service Interface

The virtual try-on service of Section 4.5 is called by the Express server. Table K.1 lists its endpoints, the data exchanged, and the retention of photographs and results.

Table K.1. Interface of the virtual try-on service (from the notebook).

Item	Request	Response
Server	FastAPI application served by uvicorn on port 8000; temporary public address created with cloudflared; API key taken from the Colab secret TRYON_API_KEY or generated at random	The address and the key are printed when the cell is run and change each time

Item	Request	Response
POST /tryon	Multipart form: person (image, at most 10 MB) and garments (JSON list of one to three items, each with a url and an optional category); header X-API-Key	job_id, status queued and the queue length; 400 for invalid garments or an unsupported category, 401 for a wrong key, 413 for a large image
GET /jobs/{id}	Header X-API-Key	status (queued, running, done or error); the path of the result when done; the error text when failed; 404 when the job is unknown or has expired
GET /results/{id}	Header X-API-Key	The try-on image as a PNG file; 404 if it is not found
GET /health	No key required	ok and the number of queued jobs
Processing	One worker thread takes the jobs from a first-in first-out queue	One generation at a time on the GPU
Retention	The uploaded photograph is deleted when the job finishes	Results are deleted after 3,600 seconds

Source: the try-on service cell of the Model 3 notebook.